

COURSE 4: PYTHON FOR DATA SCIENCE, AI & DEVELOPMENT

MODULE 1: PYTHON BASICS

4.1 COURSE INTRODUCTION

- **Python** is one of the **most popular and widely used languages** in:
 - Data Science
 - Artificial Intelligence (AI)
 - Machine Learning
 - Web Development
 - Automation & Big Data

WHY PYTHON?

- Easy to **learn and read**
- Requires **less code** than many other languages
- Huge ecosystem of **powerful libraries**
- Beginner-friendly yet powerful for experts

COURSE DESIGN

- Suitable for **beginners and experienced programmers**
 - Focuses on **core Python + real-world applications**
 - Leads into **Python Project courses** (Data Science, AI, App Dev)
-

4.2 COURSE OBJECTIVES

After completing this course, you will be able to:

- Understand **Python fundamentals**
- Work with:
 - Data types
 - Data structures
 - Control flow
 - Functions and classes
- Read & write files
- Collect data using:
 - APIs
 - Web scraping
- Apply skills in **real-world projects**

✦ **Pre-requisite** for Python Project courses

4.3 COURSE STRUCTURE (MODULES OVERVIEW)

Module 1: Python Basics

- Types
- Expressions & Variables
- String Operations

Module 2: Python Data Structures

- Lists & Tuples
- Dictionaries
- Sets

Module 3: Python Programming Fundamentals

- Conditions & Branching
- Loops

- Functions
- Exception Handling
- Objects & Classes

Module 4: Working with Data

- Reading & Writing Files
- NumPy
- Pandas

Module 5: APIs & Data Collection

- Simple APIs
 - REST APIs
 - Web Scraping
 - Final Exam
-

4.4 HELPFUL TIPS FOR COURSE COMPLETION

Step 1: Plan

- Review modules & objectives
- Set weekly goals
- Create a study timeline

Step 2: Learn Actively

- Take notes
- Download transcripts
- Highlight key concepts
- Complete all labs

Step 3: Practice & Assess

- Attempt practice quizzes
- Review mistakes
- Retake graded quizzes if needed

Step 4: Stay Accountable

- Share goals with friends/family
- Discuss concepts with peers

Step 5: Follow the Plan

- Stay consistent
 - Reward progress
-

4.5 INTRODUCTION TO PYTHON

Who Uses Python?

- Beginners
- Data Scientists
- AI Engineers
- Web Developers
- Researchers

Python Usage Statistics

- ~80% of data professionals use Python
- ~75% of data science jobs require Python

Organizations Using Python

- IBM
- Google
- NASA
- Facebook
- Amazon
- Spotify
- Reddit

Python Capabilities

- High-level, general-purpose language
- Large **standard library**
- Strong support for:
 - Data Analytics
 - Automation
 - Machine Learning
 - NLP
 - Web Scraping

Popular Python Libraries

Data Science

- Pandas
- NumPy
- SciPy
- Matplotlib

Machine Learning & AI

- TensorFlow
- PyTorch
- Keras
- Scikit-learn

NLP: Natural Language Toolkit (NLTK)

4.6 PYTHON COMMUNITY & INCLUSION

- Governed by the **Python Software Foundation**
- Strong **Code of Conduct**

- Focus on:
 - Safety
 - Inclusion
 - Diversity
-

4.7 INTRODUCTION TO JUPYTER

WHAT IS JUPYTER?

- **Jupyter Notebook** is a web-based tool for:
 - Writing code
 - Visualizing data
 - Adding text, equations, and media

SUPPORTED LANGUAGES

- Python
- R
- Julia
- Others (via kernels)

WHY JUPYTER?

- Interactive coding
 - Ideal for data exploration
 - Easy collaboration & sharing
 - Widely used in Data Science & AI
-

4.8 KEY FEATURES OF JUPYTER

- Interactive code execution
- Rich outputs (plots, tables, images)

- Integration with:
 - NumPy
 - Pandas
 - Matplotlib
 - Scikit-learn
 - TensorFlow
 - Supports collaboration via:
 - GitHub
 - Email
 - Notebook Viewer
-

4.9 GETTING STARTED WITH JUPYTER

BASIC OPERATIONS

- Run a cell → **Shift + Enter**
- Run all cells → Run → Run All
- Insert a cell → **+** **button**
- Delete a cell → Select + press **D** **twice**

WORKING WITH MULTIPLE NOTEBOOKS

- Open new notebooks via:
 - Plus button
 - File → New Notebook
- Arrange notebooks side-by-side

MARKDOWN & PRESENTATION

- Convert code cells to **Markdown**
- Add titles, explanations
- Create presentations using slide mode

SHUTTING DOWN NOTEBOOKS

- Stop kernel from sidebar
- Releases memory
- “No Kernel” confirms shutdown

HANDS-ON LAB: WRITE YOUR FIRST PROGRAM

4.10 WHAT IS A TYPE?

- A **type** defines **how Python represents and stores data**.
- Every value in Python belongs to a **specific data type**.
- You can check the type of any value using:

```
type(value)
```

4.11 COMMONLY USED DATA TYPES IN PYTHON

1. INTEGER (int)

- Represents **whole numbers**
- Can be **positive or negative**
- Examples:

```
11  
-5  
0
```

- Python supports a **very large (but finite) range** of integers

2. FLOAT (float)

- Represents **real numbers**
- Includes:
 - Decimal values
 - Numbers between integers
- Examples:

```
21.213  
0.5  
0.0001
```

✦ Floats allow **infinite precision in theory**, but in practice are limited by system memory.

3. STRING (str)

- A **sequence of characters**
- Used to represent **words or text**
- Must be enclosed in quotes
- Examples:

```
"Hello"  
"Python"  
"123"
```

CHECKING DATA TYPES

Use the `type()` function:

```
type(11)    # int  
type(21.213) # float  
type("Hello") # str
```

4.12 TYPECASTING (TYPE CONVERSION)

Typecasting means **converting one data type into another**.

COMMON TYPECASTING FUNCTIONS

- `int()`
- `float()`
- `str()`
- `bool()`

INTEGER ↔ FLOAT

```
float(2)  # 2.0
```

```
int(1.9)  # 1
```

⚠ Important:

- Casting a float to an int **removes the decimal part** (data loss).

STRING TO NUMBER

```
int("5")  # 5
```

```
float("3.14") # 3.14
```

✗ This causes an error:

```
int("Hello") # ValueError
```

✓ Only strings containing **numeric values** can be converted.

NUMBER TO STRING

```
str(10)  # "10"
```

```
str(3.14) # "3.14"
```

BOOLEAN TYPE (BOOL)

- Represents **True or False**
- Must start with **capital letters**

```
True
```

```
False
```

- Checking type:

```
type(True) # bool
```

BOOLEAN TYPECASTING RULES

Value	Converted To
True → int	1
False → int	0
1 → bool	True
0 → bool	False

Examples:

```
int(True) # 1
```

```
float(False) # 0.0
```

```
bool(1) # True
```

```
bool(0) # False
```

HANDS-ON LAB: TYPES

4.13 EXPRESSIONS

- **Expressions** are **operations that Python performs**.
- They are made up of:
 - **Operands** → values (numbers, variables)
 - **Operators** → symbols that perform operations (+, -, *, /)

EXAMPLE

`40 + 60 + 60`

- Operands: 40, 60, 60
 - Operator: +
 - Result: 160
-

4.14 ARITHMETIC OPERATORS IN PYTHON

Operator	Meaning	Example	Result
+	Addition	10 + 5	15
-	Subtraction	5 - 10	-5
*	Multiplication	5 * 5	25
/	Division	25 / 6	4.1667
//	Integer Division	25 // 6	4

Note:

- In **Python 3**, / always returns a **float**
- // performs division and **rounds down**

ORDER OF OPERATIONS (PEMDAS)

Python follows standard **mathematical conventions**:

1. Parentheses ()
2. Multiplication * and Division /
3. Addition + and Subtraction –

EXAMPLE

```
10 + 2 * 5
```

Result:

```
20
```

WITH PARENTHESES

```
(10 + 2) * 5
```

Result:

```
60
```

4.15 VARIABLES

- **Variables** store values in memory
- Assignment uses the **equals sign (=)**

EXAMPLE

```
my_variable = 1
```

- `my_variable` stores the value 1
- You can reuse it anywhere in the code

REASSIGNING VARIABLES

```
my_variable = 1  
my_variable = 10
```

- The **old value is overwritten**
- Python only remembers the **latest assignment**

STORING EXPRESSION RESULTS

```
x = 40 + 60 + 60
```

- `x` stores the result 160

```
y = x / 60
```

- `y` stores 2.6667

You can also overwrite the same variable:

```
x = x / 60
```

CHECKING VARIABLE TYPES

```
type(x)
```

- Works the same way as checking values
- Helps debug and understand data behavior

VARIABLE NAMING BEST PRACTICES

- ✓ Use **meaningful names**
- ✓ Use **underscores** to separate words (snake_case)
- ✓ Avoid single letters unless in short calculations

EXAMPLE

```
total_min = 142  
total_hour = total_min / 60
```

- Result:

```
2.367
```

✦ If total_min changes, the final result updates automatically

→ **No need to rewrite logic**

WHY VARIABLES ARE IMPORTANT

- Improve **readability**
- Make code **easy to update**
- Reduce errors
- Enable **reusability**

HANDS-ON LAB: EXPRESSION AND VARIABLES

4.16 WHAT IS A STRING?

- In **Python**, a **string** is a **sequence of characters**.
- Strings are enclosed in:
 - Double quotes " "
 - Single quotes ' '

EXAMPLES

```
"Hello"  
'Python'  
"123"  
"@# $%"
```

✓ Strings can contain:

- Letters
- Digits
- Spaces
- Special characters

ASSIGNING (BINDING) STRINGS TO VARIABLES

```
name = "Michael Jackson"
```

- The string is **bound** to the variable name.

STRINGS AS ORDERED SEQUENCES

- Strings behave like **lists or tuples**
- Each character has an **index**

POSITIVE INDEXING

```
A = "Michael Jackson"  
  
A[0]  # 'M'  
  
A[6]  # 'l'  
  
A[13] # 'n'
```

NEGATIVE INDEXING

- Indexing from the end of the string

```
A[-1] # last character
```

```
A[-15] # first character
```

STRING SLICING

Basic Slicing

```
A[start:end]
```

- start → included
- end → excluded

With Stride (Step)

```
A[::2]
```

- ✓ Selects **every second character**

```
A[0:4:2]
```

- ✓ Selects every second character **up to index 4**

LENGTH OF A STRING

Use len():

```
len(A)
```

Example:

```
len("Michael Jackson") # 15
```

STRING CONCATENATION

- Combine strings using +

```
first = "Michael"
```

```
last = "Jackson"
```

```
full = first + " " + last
```

Result:

```
"Michael Jackson"
```

STRING REPLICATION

- Multiply string by a number

```
A = "Hi "
```

```
A * 3
```

Result:

```
"Hi Hi Hi "
```

STRING IMMUTABILITY

- **Strings are immutable**
- You **cannot modify** characters directly

✗ Invalid:

```
A[0] = 'm'
```

✓ Correct approach:

```
B = A + " is the best"
```

- Creates a **new string**

ESCAPE SEQUENCES

Escape sequences begin with a **backslash (\)**

Escape Meaning

<code>\n</code>	New line
<code>\t</code>	Tab
<code>\\</code>	Backslash

Example

```
print("Hello\nWorld")
```

Output:

```
Hello  
World
```

Raw Strings (r"")

- Prevent escape sequence interpretation
- Useful for **file paths & regex**

Regular String

```
"C:\new_folder\file.txt"
```

✗ Incorrect output due to \n

Raw String

```
r"C:\new_folder\file.txt"
```

✓ Interpreted literally

4.17 STRING METHODS

- Methods return a **new string**
- Original string remains unchanged

upper()

```
A = "python"  
B = A.upper()
```

Result:

```
"PYTHON"
```

replace()

```
A.replace("Jackson", "is the best")
```

find()

```
A.find("Jack")
```

- Returns **starting index**
 - Returns -1 if substring not found
-

4.18 FORMAT STRINGS IN PYTHON

1 f-Strings (Recommended)

```
name = "John"
age = 30
print(f'My name is {name} and I am {age} years old.')
```

✓ Supports expressions:

```
f'{x + y}'
```

2 str.format()

```
"My name is {} and I am {} years old.".format(name, age)
```

3 % Operator (Legacy)

```
"My name is %s and I am %d years old." % (name, age)
```

Specifier	Meaning
%s	String
%d	Integer

✦ **f-strings** are fastest, cleanest, and preferred.

[HANDS-ON LAB: STRING OPERATIONS](#)

CHEAT SHEET: PYTHON BASICS

Package/Method	Description	Code Example
Comments	Comments are lines of text that are ignored by the Python interpreter when executing the code</td>	<pre>1 # This is a comment</pre>
Concatenation	Combines (concatenates) strings.	Syntax: <pre>1 concatenated_string = string1 + string2</pre> Example: <pre>1 result = "Hello" + " John"</pre>
Data Types	- Integer - Float - Boolean - String	Example: <pre>1 x=7 2 # Integer Value 3 y=12.4 4 # Float Value 5 is_valid = True 6 # Boolean Value 7 is_valid = False 8 # Boolean Value 9 F_Name = "John" 10 # String Value</pre>
Indexing	Accesses character at a specific index.	Example: <pre>1 my_string="Hello" 2 char = my_string[0]</pre>
len()	Returns the length of a string.	Syntax: <pre>1 len(string_name)</pre> Example: <pre>1 my_string="Hello" 2 length = len(my_string)</pre>
lower()	Converts string to lowercase.	Example: <pre>1 my_string="Hello" 2 uppercase_text = my_string.lower()</pre>
print()	Prints the message or variable inside '()'.	Example: <pre>1 print("Hello, world") 2 print(a+b)</pre>
Python Operators	- Addition (+): Adds two values together. - Subtraction (-): Subtracts one value from another. - Multiplication (*): Multiplies two values. - Division (/): Divides one value by another, returns a float. - Floor Division (//): Divides one value by another, returns the quotient as an integer. - Modulo (%): Returns the remainder after division.	Example: <pre>1 x = 9 y = 4 2 result_add= x + y # Addition 3 result_sub= x - y # Subtraction 4 result_mul= x * y # Multiplication 5 result_div= x / y # Division 6 result_fdiv= x // y # Floor Division 7 result_mod= x % y # Modulo</pre>
replace()	Replaces substrings.	Example: <pre>1 my_string="Hello" 2 new_text = my_string.replace("Hello", "Hi")</pre>
Slicing	Extracts a portion of the string.	Syntax: <pre>1 substring = string_name[start:end]</pre> Example: <pre>1 my_string="Hello" substring = my_string</pre>
split()	Splits string into a list based on a delimiter.	Example: <pre>1 my_string="Hello" 2 split_text = my_string.split(",")</pre>
strip()	Removes leading/trailing whitespace.	Example: <pre>1 my_string="Hello" 2 trimmed = my_string.strip()</pre>

upper()	Converts string to uppercase.	Example: <pre>1 my_string="Hello" 2 uppercase_text = my_string.upper()</pre>
Variable Assignment	Assigns a value to a variable.	Syntax: <pre>1 variable_name = value</pre> Example: <pre>1 name="John" # assigning John to variable name 2 x = 5 # assigning 5 to variable x</pre>

Welcome! This alphabetized glossary contains many of the terms you'll find within this course. This comprehensive glossary also includes additional industry-recognized terms not used in course videos. These terms are important for you to recognize when working in the industry, participating in user groups, and participating in other certificate programs.

Term	Definition
AI	AI (artificial intelligence) is the ability of a digital computer or computer-controlled robot to perform tasks commonly associated with intelligent beings.
Application development	Application development, or app development, is the process of planning, designing, creating, testing, and deploying a software application to perform various business operations.
Arithmetic Operations	Arithmetic operations are the basic calculations we make in everyday life like addition, subtraction, multiplication and division. It is also called as algebraic operations or mathematical operations.
Array of numbers	Set of numbers or objects that follow a pattern presented as an arrangement of rows and columns to explain multiplication.
Assignment operator in Python	Assignment operator is a type of Binary operator that helps in modifying the variable to its left with the use of its value to the right. The symbol used for assignment operator is "=".
Asterisk	Symbol "*" used to perform various operations in Python.
Backslash	A backslash is an escape character used in Python strings to indicate that the character immediately following it should be treated in a special way, such as being treated as escaped character or raw string.
Boolean	Denoting a system of algebraic notation used to represent logical propositions by means of the binary digits 0 (false) and 1 (true).
Colon	A colon is used to represent an indented block. It is also used to fetch data and index ranges or arrays.
Concatenate	Link (things) together in a chain or series.
Data engineering	Data engineers are responsible for turning raw data into information that an organization can understand and use. Their work involves blending, testing, and optimizing data from numerous sources.
Data science	Data Science is an interdisciplinary field that focuses on extracting knowledge from data sets which are typically huge in amount. The field encompasses analysis, preparing data for analysis, and presenting findings to inform high-level decisions in an organization.
Data type	Data type refers to the type of value a variable has and what type of mathematical, relational or logical operations can be applied without causing an error.
Double quote	Symbol "" used to represent strings in Python.
Escape sequence	An escape sequence is two or more characters that often begin with an escape character that tell the computer to perform a function or command.
Expression	An expression is a combination of operators and operands that is interpreted to produce some other value.
Float	Python float () function is used to return a floating point number from a number or a string representation of a numeric value.
Forward slash	Symbol "/" used to perform various operations in Python.
Foundational	Denoting an underlying basis or principle; fundamental.
Immutable	Immutable Objects are of in-built datatypes like int, float, bool, string, Unicode, and tuple. In simple words, an immutable object can't be changed after it is created.
Integer	An integer is the number zero (0), a positive natural number (1, 2, 3, and so on) or a negative integer with a minus sign (-1, -2, -3, and so on.)
Manipulate	Is the process of modifying a string or creating a new string by making changes to existing strings.
Mathematical conventions	A mathematical convention is a fact, name, notation, or usage which is generally agreed upon by mathematicians.
Mathematical expressions	Expressions in math are mathematical statements that have a minimum of two terms containing numbers or variables, or both, connected by an operator in between.
Mathematical operations	The mathematical "operation" refers to calculating a value using operands and a math operator.

Negative indexing	Allows you to access elements of a sequence (such as a list, a string, or a tuple) from the end, using negative numbers as indexes.
Operands	The quantity on which an operation is to be done.
Operators in Python	Operators are used to perform operations on variables and values.
Parentheses	Parentheses is used to call an object.
Replicate	To make an exact copy of.
Sequence	A sequence is formally defined as a function whose domain is an interval of integers.
Single quote	Symbol "'" used to represent strings in python.
Slicing in Python	Slicing is used to return a portion from defined list.
Special characters	A special character is one that is not considered a number or letter. Symbols, accent marks, and punctuation marks are considered special characters.
Stride value	Stride is the number of bytes from one row of pixels in memory to the next row of pixels in memory.
Strings	In Python, Strings are arrays of bytes representing Unicode characters.
Substring	A substring is a sequence of characters that are part of an original string.
Type casting	The process of converting one data type to another data type is called Typecasting or Type Coercion or Type Conversion.
Types in Python	Data types are the classification or categorization of data items. It represents the kind of value that tells what operations can be performed on a particular data.
Variables	Variables are containers for storing data values.

MODULE 2: PYTHON DATA STRUCTURES

2.1 LISTS AND TUPLES (COMPOUND DATA TYPES)

In **Python**, **lists and tuples** are **compound data types**, meaning they can store **multiple values** in a **single variable**.

2.1.1 TUPLES

WHAT IS A TUPLE?

- An **ordered sequence** of elements
- Written using **parentheses ()**
- Elements are **comma-separated**
- Can store **different data types** (int, float, string, etc.)
- **Immutable** → cannot be changed after creation

Example

```
ratings = (10, 9, 6, 8, 9)
```

ACCESSING TUPLE ELEMENTS (INDEXING)

- Indexing starts at **0**
- Supports **negative indexing**

```
ratings[0]  # first element  
ratings[1]  # second element  
ratings[-1] # last element
```

TUPLE SLICING

```
ratings[0:3] # first three elements  
ratings[-2:] # last two elements
```

✦ Rule:

The **ending index is one greater** than the last element you want.

LENGTH OF A TUPLE

```
len(ratings)
```

TUPLE CONCATENATION

```
t1 = (1, 2)
t2 = (3, 4)
t3 = t1 + t2
```

TUPLE IMMUTABILITY (VERY IMPORTANT)

✗ Invalid:

```
ratings[2] = 7
```

✓ Reason:

- Tuples are **immutable**
- Variables reference the **same tuple object**

```
ratings1 = ratings
```

Both variables point to the **same tuple**, but it **cannot be changed anyway**

SORTING A TUPLE

- Tuples cannot be sorted directly
- Use `sorted()` → returns a **new list**

```
sorted(ratings)
```

NESTED TUPLES

Tuples can contain **other tuples** (nesting)

```
nt = (1, 2, ("pop", "rock"), (3, 4), "disco")
```

Accessing nested elements:

```
nt[2][1]  # "rock"
nt[3][0]  # 3
```

✦ Think of nested tuples as a **tree structure**

2.1.2 LISTS

What is a List?

- An **ordered sequence**
- Written using **square brackets []**
- **Mutable** → elements can be changed
- Can store **mixed data types**
- Can be **nested** (lists inside lists, tuples, etc.)

Example

```
L = ["HardRock", 10, 1.2]
```

ACCESSING LIST ELEMENTS

```
L[0]    # first element  
L[1]    # second element  
L[-1]   # last element
```

✓ Same indexing rules as tuples

LIST SLICING

```
L[1:3]  
L[-2:]
```

LIST CONCATENATION

```
L1 = ["a", "b"]  
L2 = ["c", "d"]  
L3 = L1 + L2
```

LIST MUTABILITY (KEY DIFFERENCE)

✓ Lists **can be changed**

```
L[0] = "Pop"
```

LIST METHODS

extend()

- Adds multiple elements
- Modifies the **original list**

```
L.extend(["Jazz", "Rock"])
```

append()

- Adds **one element**
- Even if it's a list, it is added as **one item**

```
L.append(["Classical"])
```

DELETING ELEMENTS

```
del L[0]    # delete first element  
del L[1]    # delete second element
```

CONVERTING STRING TO LIST (SPLIT())

```
s = "A B C"  
s.split()
```

Result:

```
["A", "B", "C"]
```

Using delimiter:

```
s = "A,B,C"  
s.split(",")
```

ALIASING IN LISTS

```
a = ["HardRock", 10, 1.2]  
b = a
```

- a and b reference the **same list**

```
a[0] = "Banana"
```

👉 b also changes !

CLONING A LIST (AVOID SIDE EFFECTS)

```
b = a[:]
```

✓ Creates a **new copy**

- Changes to a will **not affect** b

GETTING HELP

Use the built-in help system:

```
help(list)
```

```
help(tuple)
```

🔑 KEY DIFFERENCES: LIST VS TUPLE

Feature	List	Tuple
Syntax	[]	()
Mutable	✓ Yes	✗ No
Speed	Slower	Faster
Methods	More	Fewer
Use Case	Dynamic data	Fixed data

[HANDS-ON LAB: LISTS](#)

[HANDS-ON LAB: TUPLES](#)

CHEAT SHEET: LISTS AND TUPLES

Package/Method	Description	Code Example
append()	The 'append()' method is used to add an element to the end of a list.	Syntax: <pre>1 list_name.append(element)</pre> Example: <pre>1 fruits = ["apple", "banana", "orange"] 2 fruits.append("mango") print(fruits)</pre>
copy()	The 'copy()' method is used to create a shallow copy of a list.	Example 1: <pre>1 my_list = [1, 2, 3, 4, 5] 2 new_list = my_list.copy() print(new_list) 3 # Output: [1, 2, 3, 4, 5]</pre>
count()	The 'count()' method is used to count the number of occurrences of a specific element in a list in Python.	Example: <pre>1 my_list = [1, 2, 2, 3, 4, 2, 5, 2] 2 count = my_list.count(2) print(count) 3 # Output: 4</pre>
Creating a list	A list is a built-in data type that represents an ordered and mutable collection of elements. Lists are enclosed in square brackets [] and elements are separated by commas.	Example: <pre>1 fruits = ["apple", "banana", "orange", "mango"]</pre>
del	The 'del' statement is used to remove an element from list. 'del' statement removes the element at the specified index.	Example: <pre>1 my_list = [10, 20, 30, 40, 50] 2 del my_list[2] # Removes the element at index 2 print(my_list) 3 # Output: [10, 20, 40, 50]</pre>
extend()	The 'extend()' method is used to add multiple elements to a list. It takes an iterable (such as another list, tuple, or string) and appends each element of the iterable to the original list.	Syntax: <pre>1 list_name.extend(iterable)</pre> Example: <pre>1 fruits = ["apple", "banana", "orange"] 2 more_fruits = ["mango", "grape"] 3 fruits.extend(more_fruits) 4 print(fruits)</pre>
Indexing	Indexing in a list allows you to access individual elements by their position. In Python, indexing starts from 0 for the first element and goes up to 'length_of_list - 1'.	<pre>1 my_list = [10, 20, 30, 40, 50] 2 print(my_list[0]) 3 # Output: 10 (accessing the first element) 4 print(my_list[-1]) 5 # Output: 50 (accessing the last element using negative indexing)</pre>
insert()	The 'insert()' method is used to insert an element.	Syntax: <pre>1 list_name.insert(index, element)</pre> Example: <pre>1 my_list = [1, 2, 3, 4, 5] 2 my_list.insert(2, 6) 3 print(my_list)</pre>
Modifying a list	You can use indexing to modify or assign new values to specific elements in the list.	Example: <pre>1 my_list = [10, 20, 30, 40, 50] 2 my_list[1] = 25 # Modifying the second element 3 print(my_list) 4 # Output: [10, 25, 30, 40, 50]</pre>
pop()	'pop()' method is another way to remove an element from a list in Python. It removes and returns the element at the specified index. If you don't provide an index to the 'pop()' method, it will remove and return the last element of the list by default.	Example 1: <pre>1 my_list = [10, 20, 30, 40, 50] 2 removed_element = my_list.pop(2) # Removes and returns the element at index 2 3 print(removed_element) 4 # Output: 30 5 6 print(my_list) 7 # Output: [10, 20, 40, 50]</pre> Example 2: <pre>1 my_list = [10, 20, 30, 40, 50] 2 removed_element = my_list.pop() # Removes and returns the last element 3 print(removed_element) 4 # Output: 50 5 6 print(my_list) 7 # Output: [10, 20, 30, 40]</pre>

remove()	To remove an element from a list. The 'remove()' method removes the first occurrence of the specified value.	<pre>1 my_list = [10, 20, 30, 40, 50] 2 my_list.remove(30) # Removes the element 30 3 print(my_list) 4 # Output: [10, 20, 40, 50]</pre>
reverse()	The 'reverse()' method is used to reverse the order of elements in a list	Example 1: <pre>1 my_list = [1, 2, 3, 4, 5] 2 my_list.reverse() print(my_list) 3 # Output: [5, 4, 3, 2, 1]</pre>
Slicing	You can use slicing to access a range of elements from a list.	Syntax: <pre>1 list_name[start:end:step]</pre> Example: <pre>1 my_list = [1, 2, 3, 4, 5] 2 print(my_list[1:4]) 3 # Output: [2, 3, 4] (elements from index 1 to 3) 4 5 print(my_list[:3]) 6 # Output: [1, 2, 3] (elements from the beginning up to index 2) 7 8 print(my_list[2:]) 9 # Output: [3, 4, 5] (elements from index 2 to the end) 10 11 print(my_list[::2]) 12 # Output: [1, 3, 5] (every second element)</pre>
sort()	The 'sort()' method is used to sort the elements of a list in ascending order. If you want to sort the list in descending order, you can pass the 'reverse=True' argument to the 'sort()' method.	Example 1: <pre>1 my_list = [5, 2, 8, 1, 9] 2 my_list.sort() 3 print(my_list) 4 # Output: [1, 2, 5, 8, 9]</pre> Example 2: <pre>1 my_list = [5, 2, 8, 1, 9] 2 my_list.sort(reverse=True) 3 print(my_list) 4 # Output: [9, 8, 5, 2, 1]</pre>

Tuple

Package/Method	Description	Code Example
count()	The count() method for a tuple is used to count how many times a specified element appears in the tuple.	Syntax: <pre>1 tuple.count(value)</pre> Example: <pre>1 fruits = ("apple", "banana", "apple", "orange") 2 print(fruits.count("apple")) #Counts the number of times apple is found in tuple. 3 #Output: 2</pre>
index()	The index() method in a tuple is used to find the first occurrence of a specified value and returns its position (index). If the value is not found, it raises a ValueError.	Syntax: <pre>1 tuple.index(value)</pre> Example: <pre>1 fruits = ("apple", "banana", "orange","apple") 2 print(fruits.index("apple")) #Returns the index value at which apple is present. 3 #Output: 0</pre>
sum()	The sum() function in Python can be used to calculate the sum of all elements in a tuple, provided that the elements are numeric (integers or floats).	Syntax: <pre>1 sum(tuple)</pre> Example: <pre>1 numbers = (10, 20, 5, 30) 2 print(sum(numbers)) 3 #Output: 65</pre>
min() and max()	Find the smallest (min()) or largest (max()) element in a tuple.	Example: <pre>1 numbers = (10, 20, 5, 30) 2 print(min(numbers)) 3 #Output: 5 4 print(max(numbers)) 5 #Output: 30</pre>
len()	Get the number of elements in the tuple using len().	Syntax: <pre>1 len(tuple)</pre> Example: <pre>1 fruits = ("apple", "banana", "orange") 2 print(len(fruits)) #Returns length of the tuple. 3 #Output: 3</pre>

2.2 WHAT IS A DICTIONARY?

- A **dictionary** stores data as:
 - **Keys** → act like addresses (similar to indexes)
 - **Values** → contain the actual data
- Unlike lists:
 - Keys **do not have to be integers**
 - Keys are usually **strings**

DICTIONARY SYNTAX

- Dictionaries are written using **curly braces { }**
- Each key-value pair is written as:
- key : value
- Pairs are separated by commas

Example

```
album_year = {  
    "Back in Black": 1980,  
    "The Dark Side of the Moon": 1973,  
    "The Bodyguard": 1992,  
    "Thriller": 1982  
}
```

DICTIONARY RULES

Keys

- Must be **immutable** (string, integer, tuple)
- Must be **unique**
- Usually written as **strings**

Values

- Can be **immutable or mutable**
- **Duplicates allowed**

ACCESSING VALUES USING KEYS

- Use **square brackets []**
- Provide the **key**

```
album_year["Back in Black"]    # 1980  
album_year["The Dark Side of the Moon"] # 1973
```

✖ If the key does not exist → **KeyError**

ADDING A NEW ENTRY

```
album_year["Graduation"] = 2007
```

✔ Adds a **new key-value pair**

DELETING AN ENTRY

```
del album_year["Thriller"]
```

✔ Removes the key **and its value**

CHECKING IF A KEY EXISTS

Use the **in** keyword:

```
"Thriller" in album_year
```

- Returns **True** if key exists
- Returns **False** otherwise

✖ **Note:**

in checks **keys**, not values

GETTING ALL KEYS

```
album_year.keys()
```

- Returns a **list-like object** of all keys

GETTING ALL VALUES

```
album_year.values()
```

- Returns a **list-like object** of all values

DICTIONARY VISUALIZATION

Key	Value
Back in Black	1980
The Dark Side of the Moon	1973
The Bodyguard	1992
Graduation	2007

KEY DIFFERENCES: DICTIONARY VS LIST

Feature	List	Dictionary
Indexing	Integer	Key-based
Order	Ordered	Ordered (Python ≥ 3.7)
Lookup Speed	Slower	Faster
Use Case	Sequential data	Labeled data

HANDS-ON LAB: DICTIONARIES

2.3 WHAT IS A SET?

- A **set** is:
 - A **collection** of elements
 - **Unordered** → no fixed position or index
 - Contains **only unique elements**
- Unlike lists and tuples:
 - Sets **do not allow duplicates**
 - Elements have **no index**

CREATING A SET

- Use **curly braces** { }

Example

```
album_set = {"ACDC", "BackInBlack", "Thriller", "ACDC"}
```

✓ Output:

```
{'ACDC', 'BackInBlack', 'Thriller'}
```

✗ Duplicate values are **automatically removed**

TYPECASTING: LIST → SET

- Convert a list to a set using set()
- Useful for **removing duplicates**

```
album_list = ["ACDC", "ACDC", "BackInBlack", "Thriller"]  
album_set = set(album_list)
```

✓ Result:

```
{'ACDC', 'BackInBlack', 'Thriller'}
```

IMPORTANT PROPERTIES OF SETS

Property	Description
Ordered	✗ No
Indexed	✗ No
Duplicates	✗ Not allowed
Mutable	✓ Yes
Use Case	Unique values, math operations

ADDING ELEMENTS TO A SET

Use the **add()** method:

```
album_set.add("InSync")
```

- ✓ Adds element if not present
- ✗ Adding same element again → no change

REMOVING ELEMENTS FROM A SET

Use the **remove()** method:

```
album_set.remove("InSync")
```

- ✓ Removes the element
- ⚠ Error if element does not exist

CHECKING MEMBERSHIP

Use the **in** keyword:

```
"ACDC" in album_set    # True  
"Who" in album_set     # False
```

- ✓ Checks if element exists in the set

2.3.1 MATHEMATICAL SET OPERATIONS

Sets support **mathematical operations**, often visualized using **Venn diagrams**.

◆ Intersection (AND)

- Elements common to **both sets**
- Operator: &

```
AlbumSet1 & AlbumSet2
```

- ✓ Result: elements present in **both set**

◆ Union (OR)

- All elements from **both sets**
- Operator: |

AlbumSet1 | AlbumSet2

✓ Result: combined unique elements

◆ Subset

- Checks if all elements of one set exist in another
- Method: `issubset()`

AlbumSet3.issubset(AlbumSet1)

✓ Returns True or False

SET OPERATIONS SUMMARY

Operation	Symbol / Method	Meaning
Add	<code>add()</code>	Insert element
Remove	<code>remove()</code>	Delete element
Membership	<code>in</code>	Check existence
Intersection	<code>&</code>	Common elements
Union	<code>`</code>	<code>`</code>
Subset	<code>issubset()</code>	Containment

🔑 REAL-WORLD USE CASES

- Removing duplicates from data
- Comparing datasets
- Data cleaning
- Membership testing
- Mathematical operations

HANDS-ON LAB: SETS

CHEATSHEET: DICTIONARIES & SETS

Package/Method	Description	Code Example
Creating a Dictionary	A dictionary in Python is a built-in data structure that stores information in key-value pairs. Each key must be unique, and it is used to access its corresponding value. Dictionaries are enclosed in curly braces {}, and each pair is separated by a comma.	Example: <pre>1 dict_name = {} #Creates an empty dictionary 2 person = { "name": "John", "age": 30, "city": "New York"}</pre>
Accessing Values	You can access the values in a dictionary using their corresponding <code>keys</code> .	Syntax: <pre>1 Value = dict_name["key_name"]</pre> Example: <pre>1 name = person["name"] 2 age = person["age"]</pre>
Add or modify	You can add or modify elements in a dictionary by assigning a value to a key using the assignment operator =. When you assign a value to a key that does not already exist, Python automatically creates a new key-value pair and adds it to the dictionary. However, if the key already exists, the existing value will be replaced with the new one.	Syntax: <pre>1 dict_name[key] = value</pre> Example: <pre>1 person["country"] = "USA" # A new entry will be created. 2 person["city"] = "Chicago" # update the existing value for the same key</pre>
del	Removes the specified key-value pair from the dictionary. Raises a <code>KeyError</code> if the key does not exist.	Syntax: <pre>1 del dict_name[key]</pre> Example: <pre>1 del person["country"]</pre>
key existence	You can check for the existence of a key in a dictionary using the <code>in</code> keyword	Example: <pre>1 if "name" in person: 2 print("Name exists in the dictionary.")</pre>
keys()	Retrieves all keys from the dictionary and converts them into a list. Useful for iterating or processing keys using list methods.	Syntax: <pre>1 keys_list = list(dict_name.keys())</pre> Example: <pre>1 person_keys = list(person.keys())</pre>
values()	Extracts all values from the dictionary and converts them into a list. This list can be used for further processing or analysis.	Syntax: <pre>1 values_list = list(dict_name.values())</pre> Example: <pre>1 person_values = list(person.values())</pre>
Items()	Retrieves all key-value pairs as tuples and converts them into a list of tuples. Each tuple consists of a key and its corresponding value.	Syntax: <pre>1 items_list = list(dict_name.items())</pre> Example: <pre>1 info = list(person.items())</pre>

Sets		
Package/Method	Description	Code Example
add()	Add elements to a set. You can use the <code>add()</code> method to insert a new element into a set. If the element already exists, it will not be added again, since sets automatically store only unique values.	Syntax: <pre>1 set_name.add(element)</pre> Example: <pre>1 fruits.add("mango")</pre>
Defining Sets	A set in Python is an unordered collection that stores unique elements, meaning it automatically removes duplicates. Sets are enclosed in curly braces <code>{ }</code> and are commonly used when you need to store a group of distinct values or perform mathematical operations such as union, intersection, and difference. Because sets are unordered, the elements have no fixed position or index, and their order may change each time you print or iterate over them.	Example: <pre>1 empty_set = set() #Creating an Empty Set 2 fruits = {"apple", "banana", "orange"} 3 colors = ("orange", "red", "green")</pre> Note: These two sets will be used in the examples that follow.
in	Checks if an element exists in a set. Use the <code>in</code> keyword to verify whether a specific element is present in a set. It returns <code>True</code> if the element exists, and <code>False</code> otherwise.	Syntax: <pre>1 element in set_name</pre> Example: <pre>1 "banana" in fruits</pre>
issubset()	Check if one set is a subset of another. The <code>issubset()</code> method is used to determine whether all elements of one set exist within another set. If every element of the current set is found in the specified set, it returns <code>True</code> ; otherwise, it returns <code>False</code> .	Syntax: <pre>1 is_subset = set1.issubset(set2)</pre> Example: <pre>1 is_subset = fruits.issubset(colors)</pre>
issuperset()	Check if one set is a superset of another. The <code>issuperset()</code> method is used to verify whether the current set contains all elements of another set. It returns <code>True</code> if every element of the specified set exists within the current set; otherwise, it returns <code>False</code> .	Syntax: <pre>1 is_superset = set1.issuperset(set2)</pre> Example: <pre>1 is_superset = colors.issuperset(fruits)</pre>
remove()	Remove a specific element from a set. The <code>remove()</code> method is used to delete a particular element from a set. If the element you try to remove does not exist, Python will raise a <code>KeyError</code>, which means you must ensure the element is present before calling this method. This method is useful when you are certain that the item you want to remove is already in the set.	Syntax: <pre>1 set_name.remove(element)</pre> Example: <pre>1 fruits.remove("banana")</pre>
Set Operations	Perform various operations on sets: <code>union</code> , <code>intersection</code> , <code>difference</code> , <code>symmetric difference</code> .	Syntax: <pre>1 union_set = set1.union(set2) 2 intersection_set = set1.intersection(set2) 3 difference_set = set1.difference(set2) 4 sym_diff_set = set1.symmetric_difference(set2)</pre> Example: <pre>1 combined = fruits.union(colors) 2 common = fruits.intersection(colors) 3 unique_to_fruits = fruits.difference(colors) 4 sym_diff = fruits.symmetric_difference(colors)</pre>

MODULE 2 GLOSSARY: PYTHON DATA STRUCTURES

Term	Definition
Aliasing	Aliasing refers to giving another name to a function or a variable.
Ampersand	A character typically "&" standing for the word "and."
Compound elements	Compound statements contain (groups of) other statements; they affect or control the execution of those other statements in some way.
Delimiter	A delimiter in Python is a character or sequence of characters used to separate or mark the boundaries between elements or fields within a larger data structure, such as a string or a file.
Dictionaries	A dictionary in Python is a data structure that stores a collection of key-value pairs, where each key is unique and associated with a specific value.
Function	A function is a block of code, defining a set procedure, which is executed only when it is called.
Immutable	Immutable Objects are of in-built datatypes like int, float, bool, string, Unicode, and tuple. In simple words, an immutable object can't be changed after it is created.
Intersection	The intersection of two sets is a new set containing only the elements that are present in both sets.
Keys	The keys () method in Python Dictionary, returns a view object that displays a list of all the keys in the dictionary in order of insertion using Python.
Lists	A list is any list of data items, separated by commas, inside square brackets.
Logic operations	In Python, logic operations refer to the use of logical operators such as "and," "or," and "not" to perform logical operations on Boolean values (True or False).
Mutable	Mutable objects in Python are objects whose values can be changed after they are created. These objects allow modifications such as adding, removing, or altering elements without creating a new object.
Nesting	A nested function is simply a function within another function and is sometimes called an "inner function".
Ratings in python	Ratings in Python typically refer to a numerical or qualitative measure assigned to something to indicate its quality, performance, or value.
Set operations	Set operations in Python refer to mathematical operations performed on sets, which are unordered collections of unique elements.
Sets in python	A set is an unordered collection of unique elements.
Syntax	The rules that define the structure of the language for python is called its syntax.
Tuples	These are used store multiple items in a single variable.
Type casting	In python, this is converting one data type to another.
Variables	In python, a variable is a symbolic name or identifier used to store and manipulate data. Variables serve as containers for values, and these values can be of various data types, including numbers, strings, lists, and more.
Venn diagram	A Venn diagram is a graphical representation that uses overlapping circles to illustrate the relationships and commonalities between sets or groups of items.
Versatile data	Versatile data, in a general context, refers to data that can be used in multiple ways, is adaptable to different applications or purposes, and is not restricted to a specific use case.

MODULE 3: PYTHON PROGRAMMING FUNDAMENTALS

3.1 COMPARISON OPERATIONS

Comparison operators **compare two values (operands)** and return a **Boolean value**:

- True
- False

◆ EQUALITY OPERATOR (==)

Checks if two values are equal.

```
a = 6
a == 7 # False
a == 6 # True
```

◆ GREATER THAN (>)

```
i = 6
i > 5 # True
```

✓ Condition is true if left operand is larger.

◆ GREATER THAN OR EQUAL TO (>=)

```
i = 5
i >= 5 # True
```

✓ Includes the boundary value.

◆ LESS THAN (<)

```
i = 2
i < 6 # True
```

◆ NOT EQUAL (!=)

```
i = 2
i != 6 # True
```

✓ True when values are different.

◆ COMPARING STRINGS

```
"ACDC" == "Michael Jackson" # False
```

```
"ACDC" != "Michael Jackson" # True
```

✦ String comparison is **case-sensitive**

3.2 BOOLEAN VALUES

- Result of comparison operations
- Used for decision-making

True, False

3.3 BRANCHING

a). if statement

Branching allows execution of code **only when a condition is true**.

◆ Syntax

```
if condition:  
    statement(s)
```

✦ Indentation is **mandatory**

◆ EXAMPLE: AGE CHECK

```
age = 19  
  
if age >= 18:  
    print("You can enter")  
  
print("Move on")
```

✓ If condition is True → code inside if runs

✗ If False → skipped

b). else Statement

Runs when the if condition is **False**.

◆ Syntax

```
if condition:  
    statement(s)  
else:  
    statement(s)
```

◆ EXAMPLE

```
age = 17  
  
if age >= 18:  
    print("You can enter the ACDC concert")  
else:  
    print("Go see Meatloaf")
```

✓ Only **one block** executes

c). elif Statement (Else If)

Used to check **multiple conditions**.

◆ Syntax

```
if condition1:  
    statement(s)  
elif condition2:  
    statement(s)  
else:  
    statement(s)
```

◆ EXAMPLE: CONCERT SELECTION

```
age = 18

if age > 18:
    print("You can see ACDC")
elif age == 18:
    print("Go see Pink Floyd")
else:
    print("Go see Meatloaf")
```

Age	Output
19	ACDC
18	Pink Floyd
17	Meatloaf

3.4 LOGICAL OPERATORS

Logical operators **combine Boolean expressions**.

◆ NOT Operator (not)

Reverses the Boolean value.

```
not True  # False
not False # True
```

◆ OR Operator (or)

✓ True if **at least one** condition is true

✗ False only if **both** are false

A or B

A	B	Result
T	T	T
T	F	T
F	T	T
F	F	F

◆ **Example: Album Year (70s or 90s)**

```
album_year = 1990

if album_year < 1980 or album_year > 1989:
    print("Album made in the 70s or 90s")
```

✓ Executes because one condition is true

◆ **AND Operator (and)**

✓ True **only if both conditions are true**

A and B

A	B	Result
T	T	T
T	F	F
F	T	F
F	F	F

◆ **Example: Album Made in the 80s**

```
album_year = 1983

if album_year >= 1980 and album_year <= 1989:
    print("Album made in the 80s")
```

✓ Both conditions must be true

3.5 THE RANGE() FUNCTION

The range() function generates an **ordered sequence of numbers**.

◆ range(n)

- Starts from 0
- Ends at n-1

```
range(3) → 0, 1, 2
```

◆ range(start, stop)

- Starts at start
- Ends at stop - 1

```
range(10, 15) → 10, 11, 12, 13, 14
```

✚ Note (Python 3):

- range() does **not** create a list automatically
 - It creates a **range object** (efficient & memory-friendly)
-

3.6 FOR LOOPS

A **for loop** repeats a block of code a **fixed number of times**.

◆ General Syntax

```
for variable in sequence:  
    statement(s)
```

LOOPING USING INDEX (RANGE + LIST)

Used when you need **index access**.

◆ Example: Replace All Colors with "white"

```
squares = ["red", "yellow", "green", "blue", "black"]  
  
for i in range(len(squares)):  
    squares[i] = "white"
```

◆ How it works

- `i` starts from 0
- Runs once for each element
- Index increases automatically ($i = i + 1$)

LOOPING DIRECTLY OVER A LIST

Used when **index is not needed**.

◆ Example

```
squares = ["red", "yellow", "green"]  
  
for square in squares:  
    print(square)
```

◆ Output

```
red  
yellow  
green
```

✓ Cleaner and more readable than index-based looping

3.7 enumerate() FUNCTION

Used when you need **both index and value**.

◆ Syntax

```
for index, value in enumerate(sequence):  
    statement(s)
```

◆ Example

```
squares = ["red", "yellow", "green"]  
  
for i, square in enumerate(squares):  
    print(i, square)
```

◆ Output

```
0 red  
1 yellow  
2 green
```

✦ Very useful in **data processing & analytics**

3.8 WHILE LOOPS

A **while loop** runs as long as a condition is **True**.

◆ General Syntax

```
while condition:  
    statement(s)
```

⚠ Condition must eventually become false to avoid **infinite loops**

WHILE LOOP EXAMPLE (STOPPING ON CONDITION)

◆ Problem

- Copy only **orange squares**
- Stop when a non-orange square appears

◆ Code Example

```
squares = ["orange", "orange", "purple", "orange"]  
new_squares = []  
  
i = 0  
while squares[i] == "orange":  
    new_squares.append(squares[i])  
    i += 1
```

◆ Execution Flow

1. Start at index 0
2. Copy orange squares
3. Stop when "purple" is found
4. Loop exits automatically

3.9 FOR LOOP VS WHILE LOOP

Feature	For Loop	While Loop
Runs	Fixed number of times	Until condition fails
Best for	Known iterations	Unknown iterations
Common use	Lists, ranges	Conditional repetition

[HANDS-ON LAB: LOOPS](#)

TOPIC: BUILT-IN FUNCTIONS, USER-DEFINED FUNCTIONS & SCOPE

In **Python**, a **function** is a **reusable block of code** that:

- Takes **input (parameters)**
- Performs some operation
- Returns **output** (optional)

Functions help make code:

- ✓ Shorter
 - ✓ Reusable
 - ✓ Easier to read and maintain
-

3.10 WHY USE FUNCTIONS?

Without functions:

- Same code is written repeatedly
- Code becomes long and hard to maintain

With functions:

- Write once, reuse many times
- Call the function whenever needed

✚ Think of a function as a **machine**:

Input → Function → Output

3.11 BUILT-IN FUNCTIONS

Python provides many **predefined functions**.

You only need to know **what they do**, not how they are implemented.

◆ **len()** – LENGTH OF A SEQUENCE OR COLLECTION

```
L = [10, 20, 30, 40, 50, 60, 70, 80]
l = len(L)
```

✓ **Output:** 8

Works with:

- Strings
- Lists
- Tuples
- Dictionaries
- Sets

◆ **sum()** – SUM OF ELEMENTS IN AN ITERABLE

```
numbers = [10, 20, 30, 10]
s = sum(numbers)
```

✓ **Output:** 70

SORTING: sorted() vs list.sort()

◆ **sorted()** → Function

- Returns a **new sorted list**
- Original list remains unchanged

```
ratings = [10, 9, 6, 8, 10]
new_ratings = sorted(ratings)
```

✓ ratings unchanged

✓ new_ratings is sorted

◆ **.sort() → List Method**

- Sorts the **original list**
- No new list created

```
ratings.sort()
```

✓ ratings is modified

🔑 **KEY DIFFERENCE**

Feature	sorted()	.sort()
Creates new list	Yes	No
Modifies original	No	Yes
Type	Function	Method

3.12 DEFINING YOUR OWN FUNCTION

◆ **BASIC SYNTAX**

```
def function_name(parameter):  
    statements  
    return value
```

◆ **EXAMPLE: ADD 1 TO A NUMBER**

```
def add_one(a):  
    b = a + 1  
    return b
```

Calling the function:

```
add_one(5) # Output: 6  
add_one(10) # Output: 11
```

✦ Each function call:

- Starts fresh
- Uses the new input value

3.13 FUNCTION DOCUMENTATION (DOCSTRINGS)

Used to explain what a function does.

```
def add_one(a):  
    """  
    This function adds 1 to the input value.  
    """  
    return a + 1
```

Use:

```
help(add_one)
```

✓ Displays function description

3.14 FUNCTIONS WITH MULTIPLE PARAMETERS

```
def mult(a, b):  
    return a * b
```

Examples:

```
mult(2, 3)      # 6  
mult(10, 3.14)  # 31.4  
mult(2, "MJ")   # 'MJMJ'
```

⚠ Python allows flexible data types

✓ Useful

! Can cause logical errors → **test your code**

3.15 FUNCTIONS WITHOUT RETURN

If no return statement is used:

- Python returns **None**

```
def MJ():  
    print("Michael Jackson")  
  
x = MJ()  
  
print(x)
```

✓ Output:

```
Michael Jackson  
  
None
```

◆ EMPTY FUNCTION (PASS)

```
def no_work():
```

```
    pass
```

- ✓ Valid function
- ✓ Does nothing
- ✓ Returns None

3.16 FUNCTION WITH PRINT + RETURN

```
def add_and_print(a):  
    b = a + 1  
    print("a =", a, "b =", b)  
    return b
```

- ✓ Can **print values**
 - ✓ Can **return values**
-

3.17 USING LOOPS INSIDE FUNCTIONS

```
def print_list(stuff):  
    for i, s in enumerate(stuff):  
        print(i, s)
```

Calling:

```
print_list([10, 20, 30])
```

- ✓ Uses `enumerate()`
 - ✓ Prints index + value
-

3.18 VARIADIC PARAMETERS (*ARGS)

Allows **variable number of arguments**.

```
def print_names(*names):  
    for name in names:  
        print(name)
```

Calls:

```
print_names("MJ", "Prince", "Madonna")  
print_names("MJ", "Prince")
```

- ✓ Arguments packed into a **tuple**
-

3.19 VARIABLE SCOPE

◆ GLOBAL SCOPE

- Defined **outside functions**
- Accessible everywhere

```
x = "AC"
```

◆ LOCAL SCOPE

- Defined **inside a function**
- Exists only while function runs

```
def thriller():  
    date = 1982  
    return date
```

✓ date exists only inside thriller()

◆ SAME NAME, DIFFERENT SCOPE

```
date = 2017  
  
def thriller():  
    date = 1982  
    return date
```

✓ Local date → 1982

✓ Global date → 2017

No conflict ❌

USING GLOBAL VARIABLES INSIDE FUNCTIONS

If a variable is **not defined locally**, Python checks **global scope**.

```
rating = 9  
  
def ac_dc():  
    return rating + 1
```

✓ Uses global rating

✓ Global value remains unchanged

GLOBAL KEYWORD

Used to **modify global variables inside functions**

```
def pink_floyd():  
    global claimed_sales  
    claimed_sales = "45 million"  
  
Calling:  
pink_floyd()  
print(claimed_sales)
```

✓ Output: 45 million

⚠ Use sparingly (can make code hard to debug)

READING: EXPLORING PYTHON FUNCTIONS

HANDS-ON LAB: FUNCTIONS

TOPIC: HANDLING ERRORS GRACEFULLY IN PYTHON

In **Python**, **exception handling** allows a program to **handle runtime errors** without crashing and to continue executing safely.

3.20 WHAT IS AN EXCEPTION?

An **exception** is an **error event** that occurs during program execution.

◆ Example

- User enters a **number** when **text** is expected
- Program tries an invalid operation
- Python raises an exception instead of continuing normally

Without exception handling → **program crashes**

With exception handling → **error is handled properly**

WHY EXCEPTION HANDLING IS IMPORTANT

- ✓ Prevents program termination
- ✓ Provides meaningful error messages
- ✓ Helps programs handle unexpected user input
- ✓ Essential for real-world applications

try...except Block

The **try–except** block is the foundation of exception handling.

◆ HOW IT WORKS:

1. Python executes code inside try
2. If an error occurs → jumps to except
3. Matching exception is handled

◆ SYNTAX

```
try:  
    # code that may cause an error  
except ErrorType:  
    # code to handle the error
```

◆ EXAMPLE: FILE ERROR

```
try:  
    file = open("data.txt", "r")  
    content = file.read()  
except IOError:  
    print("Unable to open or read the data in the file.")
```

- ✓ If file cannot be read
- ✓ Python skips try block
- ✓ Executes except IOError

MULTIPLE EXCEPT BLOCKS

Different errors require different handling.

```
try:
    # risky code
except IOError:
    print("File error occurred")
except ValueError:
    print("Invalid value entered")
```

✓ Specific exceptions make debugging easier

AVOID BARE EXCEPT (BAD PRACTICE ❌)

except:

```
print("Something went wrong")
```

- ❌ Catches **all errors**
- ❌ No details about what failed
- ❌ Very hard to debug in large programs

🔴 **Best Practice:**

Always specify the exception type whenever possible.

3.21 else STATEMENT

The else block runs **only if no exception occurs**.

◆ **WHY USE IT?**

- Confirms successful execution
- Keeps success logic separate from error handling

◆ EXAMPLE

```
try:
    file = open("data.txt", "w")
    file.write("Hello World")
except IOError:
    print("Unable to write to file")
else:
    print("The file was written successfully.")
```

- ✓ Runs only if try succeeds
 - ✓ Skipped if an exception occurs
-

3.22 finally STATEMENT

The finally block **always executes**, regardless of success or failure.

◆ Used for:

- Closing files
- Releasing resources, Cleanup operations

```
try:
    file = open("data.txt", "w")
    file.write("Hello")
except IOError:
    print("File error")
else:
    print("Write successful")
finally:
    file.close()
    print("File is now closed")
```

- ✓ Executes even if an error occurs
- ✓ Ensures resources are cleaned up

COMPLETE TRY-EXCEPT-ELSE-FINALLY FLOW

Block	When it runs
try	Always (first)
except	Only if error occurs
else	Only if no error
finally	Always (last)

REAL-WORLD ANALOGY

Locked Room Example

- try → Try opening the door
- except → Handle if door is locked
- else → Enter room successfully
- finally → Close the door

READING: EXCEPTION HANDLING

HANDS-ON LAB: EXCEPTION HANDLING

TOPIC: OBJECTS, CLASSES, ATTRIBUTES, AND METHODS

In **Python**, everything is treated as an **object**.

3.23 OBJECTS IN PYTHON

Python has many built-in data types such as:

- Integers
- Floats
- Strings
- Lists
- Dictionaries
- Booleans

👉 Each of these is an **object**

♦ WHAT EVERY OBJECT HAS:

1. **Type** – what kind of object it is
2. **Internal representation** – how data is stored
3. **Methods** – functions used to interact with the object

✦ An **object** is an **instance of a particular type (class)**.

EXAMPLES OF OBJECTS AND TYPES

- Creating an integer → creates an **integer object**
- Creating a list → creates a **list object**

♦ EXAMPLE

```
a = 5      # integer object  
b = [1, 2] # list object
```

Each creation results in a **new instance** of that type.

CHECKING THE TYPE OF AN OBJECT

Use the `type()` function:

```
type([1, 2, 3]) # list
type(10)        # int
type("Python")  # str
type({"a": 1})  # dict
```

3.24 METHODS OF A CLASS

Methods are functions associated with a class and are used to **interact with object data**.

◆ EXAMPLE: LIST METHODS

```
ratings = [5, 2, 4, 1, 3]
ratings.sort()
```

- `sort()` is a **method**
- It **modifies the object**
- This changes the **state of the object**

```
ratings.reverse()
```

✦ Methods are called using:

```
object_name.method_name()
```

3.25 ENCAPSULATION CONCEPT

You **do not need to know how methods work internally**

You only need to know **how to use them**

✓ This is a key idea in **Object-Oriented Programming (OOP)**

3.26 WHAT IS A CLASS?

A **class** is a **blueprint or template** used to create objects.

- Defines **data attributes**
- Defines **methods**
- Objects are **instances of a class**

CREATING CUSTOM CLASSES

Example Classes:

- **Circle**
- **Rectangle**

DATA ATTRIBUTES

◆ Circle Class Attributes

- radius
- color

◆ Rectangle Class Attributes

- height
- width
- color

These attributes define the **state** of an object.

DEFINING A CLASS IN PYTHON

```
class Circle(object):  
    def __init__(self, radius, color):  
        self.radius = radius  
        self.color = color
```

◆ KEY POINTS:

- class keyword defines a class
- object is the parent class
- `__init__()` is the **constructor**
- self refers to the **current object**

CONSTRUCTOR (`__init__`)

- Automatically runs when an object is created
- Used to initialize data attributes

✦ Think of self as a **box containing all object data**

3.27 CREATING OBJECTS (INSTANCES)

```
RedCircle = Circle(4, "red")  
GreenCircle = Circle(2, "green")
```

Each object:

- Has its **own data**
- Belongs to the **same class**

ACCESSING DATA ATTRIBUTES

```
RedCircle.radius  
RedCircle.color
```

Changing Attributes Directly

```
RedCircle.color = "blue"
```

✓ Attribute value changes inside the object

3.28 METHODS IN A CLASS

Methods **operate on object data**.

Example: Modifying Radius

```
class Circle(object):  
    def __init__(self, radius, color):  
        self.radius = radius  
        self.color = color  
  
    def add_radius(self, r):  
        self.radius = self.radius + r
```

CALLING METHODS

```
c = Circle(2, "red")  
c.add_radius(8)
```

- ✓ self is handled automatically
 - ✓ Object state changes
 - ✓ Radius becomes 10
-

3.29 DEFAULT VALUES IN CONSTRUCTOR

Constructors can have default parameters:

```
def __init__(self, radius=3, color="red"):
```

CLASS VS OBJECT

Term	Meaning
Class	Blueprint
Object	Instance of a class
Attribute	Data inside object
Method	Function acting on object

3.30 USING dir() FUNCTION

`dir(object_name)`

- Lists **attributes and methods**
 - Names with `__` are internal
 - Focus on readable names
-

[READING: OBJECTS AND CLASSES](#)

[HANDS-ON LAB: OBJECTS AND CLASSES](#)

[PRACTICE LAB: TEXT ANALYSIS](#)

[CHEAT SHEET: PYTHON PROGRAMMING FUNDAMENTALS](#)

**[MODULE 3 GLOSSARY: PYTHON PROGRAMMING
FUNDAMENTALS](#)**

MODULE 4: WORKING WITH DATA IN PYTHON

READING FILES WITH `open()` IN PYTHON

(FILE HANDLING – READING TEXT FILES)

In **Python**, files are accessed using the built-in `open()` function, which creates a **file object**. We then use methods on this object to read data from files.

4.1 OPENING A FILE

SYNTAX

```
file_object = open(file_path, mode)
```

PARAMETERS

1. File path

- Includes file name and directory
- Example: "Example1.txt"

2. Mode

- 'r' → Read (default)
- 'w' → Write
- 'a' → Append

Example

```
file1 = open("Example1.txt", "r")
```

FILE OBJECT ATTRIBUTES

Once a file is opened, we can access its attributes.

```
file1.name # returns file name as string
```

```
file1.mode # returns mode (e.g., 'r')
```

4.2 CLOSING A FILE

You should **always close a file** after use to free system resources.

```
file1.close()
```

⚠ Forgetting to close files can cause memory or file-access issues.

USING THE WITH STATEMENT (BEST PRACTICE)

The with statement **automatically closes the file**, even if an error occurs.

EXAMPLE

```
with open("Example1.txt", "r") as file1:  
    file_stuff = file1.read()
```

- ✓ File is open only inside the indented block
 - ✓ File closes automatically after the block
-

4.3 READING FILE CONTENT

◆ read() Method

- Reads **entire file** as a single string

```
file_stuff = file1.read()  
print(file_stuff)
```

✚ New lines appear as \n (newline character)

4.4 CHECKING FILE STATUS

Inside with block:

```
file1.closed # False  
  
Outside with block:  
file1.closed # True
```

- ✗ You **cannot read** from the file once it is closed
- ✓ You **can still use stored content** outside the block

4.5 READING LINES AS A LIST

◆ `readlines()` Method

- Reads file line-by-line
- Stores each line as an element in a list

```
lines = file1.readlines()
```

Example structure:

```
[  
    "First line\n",  
    "Second line\n",  
    "Third line\n"  
]
```

READING ONE LINE AT A TIME

◆ `readline()` Method

- Reads **one line per call**

```
line1 = file1.readline()
```

```
line2 = file1.readline()
```

✓ Each call moves the file pointer forward

READING LINES USING A LOOP

```
with open("Example1.txt", "r") as file1:
```

```
    for line in file1:
```

```
        print(line)
```

✓ Best way to read large files

✓ Memory efficient

READING SPECIFIC NUMBER OF CHARACTERS

You can specify **how many characters** to read.

Example

```
file1.read(4)  # reads first 4 characters  
file1.read(16) # reads next 16 characters  
file1.read(9)  # reads next 9 characters
```

✦ Each read continues from the **current position** in the file

CHARACTER GRID CONCEPT

- File text is treated as a **sequence of characters**
- Each read operation moves forward through this sequence

READING: READING FILES WITH OPEN

HANDS-ON LAB: READING FILES WITH OPEN

WRITING FILES WITH `open()` IN PYTHON

(File Handling – Writing & Appending Text Files)

Python allows us to **create, write, append, and copy files** using the built-in `open()` function along with the `write()` method.

4.6 OPENING A FILE FOR WRITING

To write data to a file, we open it in **write mode ('w')**.

SYNTAX

```
file_object = open(file_path, "w")
```

IMPORTANT POINTS

- If the file **already exists**, it will be **overwritten**
- If the file **does not exist**, Python will **create it**

USING with STATEMENT (RECOMMENDED)

Just like reading files, using with is best practice.

```
with open("Example2.txt", "w") as file1:  
    file1.write("This is line A\n")
```

- ✓ File automatically closes after the block
 - ✓ Safer and cleaner code
-

4.7 WRITING DATA USING write()

◆ write() Method

- Writes text to a file
- Accepts **only strings**
- Does **not** add a newline automatically

EXAMPLE

```
with open("Example2.txt", "w") as file1:  
    file1.write("This is line A\n")  
    file1.write("This is line B")
```

✚ \n is used to move to a new line

WRITING MULTIPLE LINES USING A LIST

You can write **each element of a list** into a file using a loop.

EXAMPLE

```
lines = ["This is line A\n", "This is line B\n", "This is line C\n"]  
  
with open("Example2.txt", "w") as file1:  
  
    for line in lines:  
  
        file1.write(line)
```

- ✓ Each loop iteration writes one line
 - ✓ File closes automatically after loop ends
-

4.8 APPENDING TO A FILE ('a' MODE)

Append mode allows adding content **without deleting existing data**.

KEY POINTS

- Uses mode 'a'
- Does **not create a new file**
- Writes at the **end of the file**

EXAMPLE

```
with open("Example2.txt", "a") as file1:  
  
    file1.write("This is line C\n")
```

DIFFERENCE BETWEEN WRITE & APPEND MODES

Mode	Action
'w'	Overwrites existing file
'a'	Adds content at the end

4.9 COPYING ONE FILE TO ANOTHER

We can copy content from one file to another using **read + write**.

STEPS

1. Open source file in read mode
2. Open destination file in write mode
3. Use a loop to copy line by line

EXAMPLE

```
with open("Example1.txt", "r") as readfile:
    with open("Example3.txt", "w") as writefile:
        for line in readfile:
            writefile.write(line)
```

- ✓ First iteration copies first line
- ✓ Continues until end of file
- ✓ Both files close automatically

READING: WRITING FILES WITH OPEN

HANDS-ON LAB: WRITING FILES WITH OPEN

PANDAS: LOADING DATA

(Introduction to Pandas & DataFrames)

4.10 WHAT IS PANDAS?

- **Pandas** is a popular **Python library** used for **data analysis and data manipulation**
- It provides **pre-written functions and classes** to work with structured data
- Works efficiently even with **millions of records**

IMPORTING PANDAS

BASIC IMPORT

```
import pandas
```

USING ALIAS (RECOMMENDED)

```
import pandas as pd
```

- ✓ **pd** is the **standard abbreviation**
 - ✓ Makes code shorter and readable
 - ✓ Any alias name can be used, but **pd** is preferred
-

4.11 LOADING CSV FILES INTO PANDAS

CSV Files

- CSV = **Comma Separated Values**
- Common format for storing tabular data

Syntax

```
df = pd.read_csv("file_path.csv")
```

Explanation

- `read_csv()` is a **built-in Pandas function**
- File path is passed as an argument
- Data is loaded into a **DataFrame**
- `df` is a common variable name for DataFrame

VIEWING DATA USING `head()`

```
df.head()
```

- ✓ Displays **first 5 rows** of the DataFrame
- ✓ Useful for quick inspection of data

LOADING EXCEL FILES

```
df = pd.read_excel("file_path.xlsx")
```

- ✓ Similar process as CSV
 - ✓ Result is also a DataFrame
-

4.12 WHAT IS A DATAFRAME?

- A 2D table-like structure
- Consists of:
 - Rows
 - Columns
- Similar to an Excel spreadsheet or SQL table

CREATING A DATAFRAME FROM A DICTIONARY

Dictionary Structure

- Keys → Column names
- Values → List of row values

Example

```
data = {  
    "Artist": ["A", "B", "C"],  
    "Released": [1980, 1973, 1992]  
}  
  
df = pd.DataFrame(data)
```

- ✓ Keys become column headers
- ✓ Lists become column values

SELECTING COLUMNS FROM A DATAFRAME

Single Column (as DataFrame)

```
df[["Artist"]]
```

Multiple Columns

```
df[["Artist", "Released"]]
```

✦ **Double brackets** return a DataFrame

✦ **Single brackets** return a Series

ACCESSING DATA USING **iloc**

iloc is **index-based** (integer positions)

Syntax

```
df.iloc[row_index, column_index]
```

Examples

```
df.iloc[0, 0] # First row, first column  
df.iloc[1, 0] # Second row, first column  
df.iloc[0, 2] # First row, third column  
df.iloc[1, 2] # Second row, third column
```

ACCESSING DATA USING COLUMN NAMES

```
df["Artist"][0]  
df["Artist"][1]  
df["Released"][0]
```

✓ Uses column name + row index

USING **loc** (LABEL-BASED ACCESS)

- Used when **index is not an integer**
- Works with **row labels and column names**

Example

```
df_new.index = ["A", "B", "C"]  
  
df_new.loc["A", "Artist"]  
df_new.loc["B", "Artist"]  
df_new.loc["A", "Released"]
```

SLICING DATAFRAMES

Using `iloc`

```
Z = df.iloc[0:2, 0:3]
```

- ✓ First 2 rows
- ✓ First 3 columns

Using `loc`

```
Z = df.loc[0:1, "Artist":"Released"]
```

- ✓ Selects rows
 - ✓ Selects columns between specified labels
-

PANDAS: WORKING WITH AND SAVING DATA

4.13 FINDING UNIQUE VALUES

`unique()` Method

```
df["Released"].unique()
```

- ✓ Returns **all unique values** in a column
- ✓ Efficient for large datasets

FILTERING DATA USING CONDITIONS

Example: Albums released after 1979

Step 1: Condition

```
df["Released"] > 1979
```

- ✓ Returns a **Boolean Series**
- ✓ True if condition matches
- ✓ False otherwise

Step 2: Apply Filter

```
df1 = df[df["Released"] > 1979]
```

- ✓ Creates a **new DataFrame**
- ✓ Contains only filtered rows

4.14 SAVING DATAFRAMES TO CSV

Using `to_csv()`

```
df1.to_csv("new_data.csv")
```

- ✓ Saves DataFrame as CSV
- ✓ .csv extension is required
- ✓ Other formats are also supported

READING: PANDAS

PRACTICE LAB: SELECTING DATA IN A DATAFRAME

HANDS ON LAB: LOADING DATA WITH PANDAS

NUMPY NOTES (ONE-DIMENSIONAL & TWO-DIMENSIONAL)

4.15 WHAT IS NUMPY?

- NumPy = **N**umerical **P**ython
- Used for **scientific computing**
- Faster and more memory-efficient than Python lists
- Core object: **ndarray**
- Foundation of **pandas**

◆ ONE-DIMENSIONAL NUMPY (1D ARRAYS)

4.16 CREATING A NUMPY ARRAY

```
import numpy as np

a = np.array([0, 1, 2, 3, 4])
```

DIFFERENCE: LIST VS NUMPY ARRAY

Python List	NumPy Array
Mixed data types	Same data type
Slower	Faster
More memory	Less memory

ARRAY ATTRIBUTES

```
a.size    # Number of elements
a.ndim    # Number of dimensions (rank)
a.shape   # Shape of array
a.dtype   # Data type
```

EXAMPLE:

```
print(a.size)  # 5
print(a.ndim)  # 1
print(a.shape) # (5,)
print(a.dtype) # int64
```

INDEXING & SLICING

```
a[0] = 100    # Change first element
a[4] = 0       # Change fifth element
```

SLICING

```
d = a[1:4]    # Elements from index 1 to 3
```

4.17 Vector Operations (VERY IMPORTANT)

Vector Addition

```
u = np.array([1, 0])  
v = np.array([0, 1])  
  
z = u + v
```

Vector Subtraction

```
z = u - v
```

Scalar Multiplication

```
y = np.array([1, 2, 3])  
z = 2 * y
```

HADAMARD PRODUCT (ELEMENT-WISE)

```
u = np.array([1, 2])  
v = np.array([3, 4])  
  
z = u * v
```

DOT PRODUCT

```
result = np.dot(u, v)
```

➡ Returns a **single number**

BROADCASTING

```
a = np.array([1, 2, 3])  
a + 1
```

➡ Adds 1 to **each element**

4.18 UNIVERSAL FUNCTIONS (ufuncs)

```
a.mean()  
a.max()  
a.min()  
a.sum()
```

MATHEMATICAL FUNCTIONS

```
x = np.array([0, np.pi/2, np.pi])  
y = np.sin(x)
```

Linspace

```
np.linspace(-2, 2, num=5)  
np.linspace(0, 2*np.pi, 100)
```

4.19 PLOTTING WITH NUMPY

```
import matplotlib.pyplot as plt  
import numpy as np  
  
x = np.linspace(0, 2*np.pi, 100)  
y = np.sin(x)  
  
plt.plot(x, y)  
plt.show()
```

HANDS-ON LAB: ONE DIMENSIONAL NUMPY

READING: MATRIX MATHEMATICS

◆ TWO-DIMENSIONAL NUMPY (2D ARRAYS)

4.20 CREATING A 2D ARRAY

```
a = np.array([
    [11, 12, 13],
    [21, 22, 23],
    [31, 32, 33]
])
```

ATTRIBUTES

```
a.ndim    # 2
a.shape    # (3, 3)
a.size     # 9
```

INDEXING

```
a[1][2]    # 23
a[0, 1]     # 12
```

SLICING

```
a[0, 0:2]   # First row, first two columns
a[0:2, 2]   # First two rows, last column
```

4.21 MATRIX OPERATIONS

MATRIX ADDITION

```
X = np.array([[1, 2], [3, 4]])
Y = np.array([[5, 6], [7, 8]])
```

$$Z = X + Y$$

SCALAR MULTIPLICATION

$$Z = 2 * Y$$

HADAMARD PRODUCT (ELEMENT-WISE)

$$Z = X * Y$$

MATRIX MULTIPLICATION

```
A = np.array([[1, 2, 3],  
              [4, 5, 6]])
```

```
B = np.array([[1, 0],  
              [0, 1],  
              [1, 1]])
```

```
C = np.dot(A, B)
```

or

$$C = A @ B$$

[HANDS-ON LAB: TWO DIMENSIONAL NUMPY](#)

[READING: BEGINNER'S GUIDE TO NUMPY](#)

[CHEAT SHEET: WORKING WITH DATA IN PYTHON](#)

**[MODULE 4 GLOSSARY: WORKING WITH DATA IN
PYTHON](#)**

MODULE 5: API'S AND DATA COLLECTION

5.1 WHAT IS AN API? (APPLICATION PROGRAMMING INTERFACE)

- An **API** allows **two software applications to communicate** with each other
- It works using **inputs (requests)** and **outputs (responses)**
- You **don't need to know how the API works internally**, only:
 - What input it expects
 - What output it returns
- APIs are similar to **functions**, but they work **across software systems**

EXAMPLE

- Your program → sends a request
 - API → processes it
 - API → sends back data as a response
-

5.2 APIS IN PYTHON (PANDAS AS AN API)

- Pandas itself is an **API**
- Many internal components of Pandas are **not written in Python**
- You interact with Pandas using:
 - DataFrames
 - Methods like `head()`, `mean()`

EXAMPLE

```
df.head()
df.mean()
```

- When you call these methods:
 - Your DataFrame communicates with the **Pandas API**
 - API processes the request

- Result is returned to you

📌 In API terminology:

- A DataFrame object is an **instance**
 - Methods act as **API calls**
-

5.3 WHAT ARE REST APIS?

- **REST** = Representational State Transfer
- REST APIs allow communication **over the internet**
- They help access:
 - Online data
 - Cloud storage
 - AI services
 - Real-time updates

REST API COMPONENTS

Term	Meaning
Client	Your program / code
Resource	Web service
Endpoint	URL used to access API
Request	Message sent by client
Response	Data returned by API

REST API COMMUNICATION FLOW

1. **Client** sends a request
2. Request is sent via **HTTP**
3. Request usually contains **JSON**
4. Web service processes the request

5. Web service sends back a **response**
6. Response is usually in **JSON format**

- ✦ JSON = JavaScript Object Notation
 - ✦ Lightweight, structured, easy to parse
-

5.4 HTTP METHODS (BASICS)

- HTTP methods tell the API **what action to perform**
 - Commonly used with REST APIs
 - Data is transferred using **HTTP messages**
-

5.6 WHY APIS ARE IMPORTANT

- Save **time and resources**
- Allow reuse of **existing functionality**
- Enable communication between different systems
- Widely used in:
 - Web apps
 - Mobile apps
 - Desktop apps
 - Cloud services

REAL-WORLD APPLICATIONS OF APIS

◆ Social Media

- Facebook, Twitter, Instagram
- Access posts, profiles, analytics

◆ E-Commerce

- Amazon, eBay
- Product catalogs, pricing, orders

◆ Weather Apps

- AccuWeather, Weather Channel
- Live weather updates

◆ Maps & Navigation

- Google Maps, Waze
- Location, traffic, directions

◆ Payment Gateways

- PayPal, Stripe
- Secure online payments

◆ Messaging Apps

- WhatsApp, Facebook Messenger
 - Messaging features and integrations
-

5.7 CRYPTOCURRENCY APIS (PYCOINGECKO)

- Cryptocurrency data changes **every minute**
- APIs are ideal for:
 - Real-time prices
 - Market cap
 - Trading analysis

COINGECKO API

- Updated every minute
- Free and widely used

PYCOINGECKO PYTHON WRAPPER

- A **wrapper** simplifies API usage
- You don't need to write raw HTTP requests
- Focus is on **data collection**, not networking

STEPS TO USE PYCOINGECKO

1. Install and import library
 2. Create a client object
 3. Call a function to request data
-

5.8 DATA RETURNED FROM API

- Response is usually:
 - **JSON**
 - Converted to **Python dictionary**
- Contains nested lists like:
 - Price
 - Market cap
 - Volume
- Each entry includes:
 - UNIX timestamp
 - Price value

📌 UNIX timestamp = time in milliseconds

5.9 CONVERTING API DATA TO DATAFRAME

- Select only required data (e.g., price)
 - Convert nested list to DataFrame
 - Columns:
 - Timestamp
 - Price
-

5.10 WORKING WITH TIME SERIES DATA

Convert Timestamp to Readable Date

`pd.to_datetime(timestamp, unit='ms')`

- Converts UNIX timestamp → human-readable date
 - New column added: date
-

5.11 CREATING CANDLESTICK DATA

- Group data by **date**
- Calculate:
 - Open (first)
 - Close (last)
 - High (max)
 - Low (min)

📌 Used for financial analysis

CANDLESTICK PLOT

- Created using **Plotly**
 - Saved as an HTML file
 - Must click “**Trust HTML**” to view plot
-

5.12 STEP-BY-STEP CORRECT CODE

1 INSTALL & IMPORT COINGECKO

```
!pip install pycoingecko  
  
from pycoingecko import CoinGeckoAPI  
  
import pandas as pd  
  
import plotly.graph_objects as go
```

2 FETCH BITCOIN PRICE DATA

```
cg = CoinGeckoAPI()

bitcoin_data = cg.get_coin_market_chart_by_id(
    id='bitcoin',
    vs_currency='usd',
    days=30
)
```

3 CONVERT TO DATAFRAME

```
data = pd.DataFrame(bitcoin_data['prices'], columns=['Timestamp', 'Price'])

data['Date'] = pd.to_datetime(data['Timestamp'], unit='ms')
```

4 PREPARE CANDLESTICK (OHLC) DATA

Important:

- first → Open
- max → High
- min → Low
- last → Close

```
candlestick_data = (
    data
    .groupby(data['Date'].dt.date)
    .agg(
        Open=('Price', 'first'),
        High=('Price', 'max'),
        Low=('Price', 'min'),
        Close=('Price', 'last')))
```

5 CREATE CANDLESTICK CHART (PLOTLY)

```
fig = go.Figure(  
    data=[  
        go.Candlestick(  
            x=candlestick_data.index,  
            open=candlestick_data['Open'],  
            high=candlestick_data['High'],  
            low=candlestick_data['Low'],  
            close=candlestick_data['Close']  
        )  
    ]  
)
```

6 LAYOUT & EXPORT

```
fig.update_layout(  
    xaxis_rangeslider_visible=False,  
    xaxis_title='Date',  
    yaxis_title='Price (USD $)',  
    title='Bitcoin Candlestick Chart Over Past 30 Days'  
)  
  
import plotly.offline as pyo  
  
pyo.plot(fig, filename='bitcoin_candlestick_graph.html', auto_open=False)
```

[READING: SOME CONTEXT ON APIS](#)

[HANDS-ON LAB: INTRODUCTION TO API](#)

REST APIS & HTTP REQUESTS – PART 1

5.13 HTTP PROTOCOL

- **HTTP (HyperText Transfer Protocol)** is a protocol used to **transfer information over the web**
- REST APIs use **HTTP messages** to communicate
- Communication happens using:
 - **Request** → sent by client
 - **Response** → sent by server

✚ When you open a website:

- Browser = **Client**
 - Website server = **Server**
 - Browser sends an **HTTP request**
 - Server sends an **HTTP response**
-

5.14 REQUEST–RESPONSE CYCLE

Request

- Client asks for a resource
- Usually sent using HTTP methods like **GET, POST**
- May include:
 - Headers
 - Body (optional)
 - URL

Response

- Sent by server
 - Includes:
 - Status code
 - Headers
 - Body (requested data)
-

5.15 UNIFORM RESOURCE LOCATOR (URL)

A **URL** is used to locate resources on the web.

Parts of a URL

1. Scheme (Protocol)

- Example: http:// or https://

2. Base URL (Internet Address)

- Example: www.ibm.com, www.gitlab.com

3. Route (Path on Server)

- Example: /images/IDSNlogo.png

✚ Full URL = Scheme + Base URL + Route

5.16 HTTP REQUEST MESSAGE (GET EXAMPLE)

Components

- **Start Line**
 - HTTP method: GET
 - Requested resource: index.html
- **Request Header**
 - Additional info (may be empty for GET)
- **Request Body**
 - Usually empty in GET requests

Request Start line	/Get/index.html HTTP/1.0
Request Header	User-Agent: python-requests/2.21.0 Accept-Encoding: gzip, deflate :

5.17 HTTP RESPONSE MESSAGE

Components

- **Start Line**
 - HTTP version (e.g., HTTP/1.0)
 - Status code (e.g., 200)
 - Description (OK)
- **Response Header**
 - Metadata about response
- **Response Body**
 - Requested data (HTML, image, text, etc.)

Response Start line	HTTP/1.0 200 OK
Response Header	Server: Apache-Cache: UNCACHEABLE
Response Body	<!DOCTYPE html> <html> <body> My First Heading My first paragraph. </body> </html>

5.18 HTTP STATUS CODES

Status Code Classes

Code Range	Meaning
100s	Informational
200s	Successful
400s	Client errors
500s	Server errors

Common Examples

- **200** → OK (Success)
- **401** → Unauthorized
- **501** → Not Implemented

1XX	Informational
100	Everything So Far Is OK
2xx	Success
200	OK
3XX	Redirection
300	Multiple Choices

4XX	Client Error
401	Unauthorized
403	Forbidden
404	Not Found
500	Server Error
501	No Implemented

5.19 HTTP METHODS

HTTP methods tell the server **what action to perform**.

Common methods:

- **GET** → Retrieve data
- **POST** → Send data
- **PUT** → Update data
- **DELETE** → Remove data

HTTP Methods	Description
GET	Retrieves data from the server
POST	Submits data to server
PUT	Updates data already on server
DELETE	Deletes data from server

5.20 PYTHON REQUESTS LIBRARY

- Popular library for working with HTTP
- Easier than urllib or httpplib
- Supports HTTP/1.1

Importing requests

```
import requests
```

5.21 GET REQUEST IN PYTHON

EXAMPLE

```
r = requests.get("https://www.ibm.com")
```

RESPONSE OBJECT (R) ATTRIBUTES

Attribute	Description
r.status_code	HTTP status code (e.g., 200)
r.headers	Response headers (dictionary)
r.text	HTML/text content
r.encoding	Encoding type
r.request.body	Body (None for GET)

CLEAN WORKING CODE

1 MAKE A GET REQUEST

```
import requests

url = 'https://www.ibm.com/'

r = requests.get(url)
```

2 STATUS CODE

```
r.status_code
```

✓ **OUTPUT:**

200

→ Means **request successful**

3 REQUEST HEADERS (SENT BY YOUR CLIENT)

`r.request.headers`

EXAMPLE OUTPUT:

```
{  
  'User-Agent': 'python-requests/2.24.0',  
  'Accept-Encoding': 'gzip, deflate',  
  'Accept': '/*/*',  
  'Connection': 'keep-alive'  
}
```

✦ **These headers tell the server:**

- Who is making the request (User-Agent)
- What formats are accepted
- Keep connection alive

4 REQUEST BODY

`r.request.body`

✓ **OUTPUT:**

None

→ Because **GET requests do not send a body**

5 RESPONSE HEADERS (SENT BY SERVER)

```
headers = r.headers  
  
headers
```

EXAMPLE OUTPUT (SHORTENED):

```
{
  'Server': 'Apache',
  'Content-Type': 'text/html; charset=UTF-8',
  'Content-Encoding': 'gzip',
  'Cache-Control': 'public, max-age=300',
  'Date': 'Thu, 19 Nov 2020 15:21:47 GMT'
}
```

These tell you:

- Server type
- Content format
- Encoding
- Cache rules
- Response time

6 ACCESS A SPECIFIC HEADER

```
headers['Date']
```

7 CONTENT TYPE

```
headers['Content-Type']
```

OUTPUT:

```
'text/html; charset=UTF-8'
```

8 ENCODING

```
r.encoding
```

OUTPUT:

```
'UTF-8'
```

9 FIRST 100 CHARACTERS OF HTML PAGE

```
r.text[:100]
```

✓ **OUTPUT:**

```
'<!DOCTYPE html>\n<html lang="en-ca" dir="ltr">\n<head>\n <meta charset="utf-8">'
```

➔ Confirms that the response is **HTML content**

🧠 **SUMMARY**

Attribute	Meaning
r.status_code	HTTP response status
r.request.headers	Headers sent by client
r.headers	Headers returned by server
r.encoding	Character encoding
r.text	Response body (HTML)

Base URL	Route
httpbin.org	/get
httpbin.org/get	

5.22 QUERY STRINGS IN GET REQUESTS

- Used to send extra data via URL
- Starts with ?
- Format: key=value
- Multiple pairs separated by &

EXAMPLE

```
/get?name=Joseph&ID=123
```

Start of Query	Parameter Name		Value		Parameter Name		Value
?	name	=	Joseph	&	ID	=	123
http://httpbin.org/get? Name=Joseph&ID=123							

5.23 GET REQUEST WITH PARAMETERS (PYTHON)

```
payload = {"name": "Joseph", "ID": "123"}
r = requests.get(url, params=payload)
```

- Parameters sent in URL
- Request body = None
- Response often returned as **JSON**

Convert JSON to Python Dictionary

```
r.json()
```

✓ CLEAN & CORRECT CODE

```
import requests

# Base URL
url_get = 'http://httpbin.org/get'

# Query parameters
payload = {
    "name": "Joseph",
    "ID": "123"
}

# Send GET request with query string
r = requests.get(url_get, params=payload)
```

GENERATED URL (QUERY STRING)

`r.url`

✅ OUTPUT:

`http://httpbin.org/get?name=Joseph&ID=123`

📌 EXPLANATION

- `params=payload` automatically converts the dictionary into a **query string**
- Format:
- `?key1=value1&key2=value2`

REQUEST BODY

`r.request.body`

✅ OUTPUT:

`None`

📌 GET requests **do not send a body** — data goes in the URL.

STATUS CODE

`r.status_code`

✅ OUTPUT:

`200`

➡ Request successful

RESPONSE TEXT (RAW JSON)

`r.text`

✅ OUTPUT:

```
{
  "args": {
    "ID": "123",
    "name": "Joseph"
  },
  "headers": {
    "Accept": "*/*",
    "Accept-Encoding": "gzip, deflate",
    "Host": "httpbin.org",
    "User-Agent": "python-requests/2.24.0",
    "X-Amzn-Trace-Id": "Root=1-5fbbd03e-106584ab42513a6d5cef39a9"
  },
  "origin": "99.228.137.181",
  "url": "http://httpbin.org/get?name=Joseph&ID=123"
}
```

📌 httpbin.org **echoes back** whatever you send — great for testing APIs.

📄 RESPONSE HEADER

```
r.headers['Content-Type']
```

✅ OUTPUT:

```
'application/json'
```

➡ Confirms response is JSON

🔄 Convert Response to Python Dictionary

```
r.json()
```

✓ OUTPUT:

```
{
  'args': {'ID': '123', 'name': 'Joseph'},
  'headers': {...},
  'origin': '99.228.137.181',
  'url': 'http://httpbin.org/get?name=Joseph&ID=123'
}
```

📌 `r.json()` converts JSON → **Python dict**

🔗 ACCESS ONLY QUERY PARAMETERS

```
r.json()['args']
```

✓ OUTPUT:

```
{'ID': '123', 'name': 'Joseph'}
```

🧠 CONCEPT SUMMARY

Concept	Explanation
params	Sends data as query string
Query String	Data appended to URL
r.url	Final request URL
r.text	Raw response
r.json()	Parsed JSON response
args	Query parameters received by server

GET VS POST (QUICK TIP)

GET	POST
Data in URL	Data in body
Visible	Hidden
params=	data= or json=

5.24 POST REQUESTS

- Sends data in **request body**, not URL
- Used to submit data to server

EXAMPLE

```
r = requests.post(url, data=payload)
```

DIFFERENCES: GET VS POST

Feature	GET	POST
Data location	URL	Request body
Request body	None	Present
URL parameters	Yes	No

CLEAN & CORRECT POST EXAMPLE

```
import requests

# POST URL
url_post = "http://httpbin.org/post"

# Payload
payload = {
    "name": "Joseph",
    "ID": "123"
}
```

```
# Send POST request  
r_post = requests.post(url_post, data=payload)
```

COMPARE POST VS GET URLS

```
print("POST request URL:", r_post.url)  
print("GET request URL:", r.url)
```

✅ OUTPUT:

```
POST request URL: http://httpbin.org/post  
GET request URL: http://httpbin.org/get?name=Joseph&ID=123
```

KEY DIFFERENCE

- **POST** → data NOT appended to URL
- **GET** → data visible in URL (query string)

COMPARE REQUEST BODIES

```
print("POST request body:", r_post.request.body)  
print("GET request body:", r.request.body)
```

✅ OUTPUT:

```
POST request body: name=Joseph&ID=123  
GET request body: None
```

EXPLANATION:

- POST sends data in **request body**
- GET sends data in **URL**, not body

ACCESS POST DATA ON SERVER SIDE

```
r_post.json()['form']
```

✅ OUTPUT:

```
{'ID': '123', 'name': 'Joseph'}
```

📌 httpbin.org places POST form data inside:

```
response['form']
```

🧠 GET VS POST — FINAL COMPARISON TABLE

Feature	GET	POST
Data location	URL	Request body
Visibility	Visible	Hidden
Body	❌ None	✅ Present
Size limit	Limited	Larger
Security	Less secure	More secure
Use case	Fetch data	Send data

Web Scraping & HTML Basics

5.25 WHAT IS WEB SCRAPING?

- Automated extraction of data from websites
- Also called:
 - Web harvesting
 - Web data extraction

Uses

- Data analysis
- Price comparison
- Content aggregation
- Research

WEB SCRAPING WORKFLOW

1. HTTP Request
2. Web Page Retrieval

3. HTML Parsing
 4. Data Extraction
 5. Data Transformation
 6. Storage
 7. Automation
-

5.26 HTML BASICS

HTML Structure

- `<html>` → Root element
- `<head>` → Metadata
- `<body>` → Visible content

Common Tags

- `<h3>` → Heading
- `<p>` → Paragraph
- `<a>` → Link

HTML TAG COMPOSITION

- Opening tag: `<tag>`
- Closing tag: `</tag>`
- Attributes:

`<a href="link">Text</a>`

HTML DOCUMENT TREE

- HTML represented as a **tree**
- Tags = nodes
- Relationships:
 - Parent
 - Child
 - Sibling

📌 Example:

- `<html>` → parent
- `<head>` & `<body>` → children & siblings

HTML TABLES

Tag	Purpose
<code><table></code>	Table
<code><tr></code>	Row
<code><th></code>	Header
<code><td></code>	Cell

5.27 TOOLS FOR WEB SCRAPING

Required Libraries

- `requests`
- `BeautifulSoup (bs4)`

FETCHING & PARSING HTML

```
import requests

from bs4 import BeautifulSoup

response = requests.get(url)

html = response.text

soup = BeautifulSoup(html, 'html.parser')
```

NAVIGATING HTML WITH BEAUTIFULSOUP

Find all anchor tags

```
links = soup.find_all('a')  
  
for link in links:  
    print(link.text)
```

- BeautifulSoup converts HTML into a tree
- Easy navigation and extraction

EXTRACTING TABLES WITH PANDAS

```
pd.read_html(url)
```

- Automatically extracts HTML tables
- Returns DataFrames
- Very useful for data analysis

[READING: WEB SCRAPING AND HTML BASICS](#)

[HANDS-ON LAB: ACCESS REST APIS & REQUEST HTTP](#)

[HANDS-ON LAB: API EXAMPLES](#)

5.28 HTML FOR WEB SCRAPING

What is HTML?

- **HTML (HyperText Markup Language)** is the standard language used to create web pages.
- Web pages contain **valuable data** (prices, tables, names, salaries, etc.).
- To extract this data using Python, we must understand **HTML structure**.

👉 Examples of data sources:

- Wikipedia
- Real estate listings

- Sports statistics
- Coding solution pages

BASIC STRUCTURE OF AN HTML PAGE

```
<!DOCTYPE html>

<html>

  <head>

    <title>Page Title</title>

  </head>

  <body>

    <h3>Player Name</h3>

    <p>Salary</p>

  </body>

</html>
```

```
<!DOCTYPE html>

<html>

  <head>

    <title>Page Title</title>

  </head>

  <body>

    <h3><b id='boldest'> LeBron James</b></h3>

    <p> Salary: $ 92,000,000 </p>

    <h3> Stephen Curry</h3>

    <p> Salary: $85,000,000 </p>

    <h3> Kevin Durant </h3>

    <p> Salary: $73,200, 000</p></body></html>
```

Key Components

- `<!DOCTYPE html>` → Declares the document as HTML
- `<html>` → Root element
- `<head>` → Meta information (title, styles, scripts)
- `<body>` → Visible content (data we usually scrape)

COMMON HTML TAGS

Tag	Meaning
<code><h1>-<h6></code>	Headings (h1 largest, h6 smallest)
<code><p></code>	Paragraph
<code></code>	Bold text
<code><a></code>	Hyperlink
<code><table></code>	Table
<code><tr></code>	Table row
<code><td></code>	Table cell
<code><th></code>	Table header

COMPOSITION OF AN HTML TAG

Example: Anchor (`<a>`) Tag

`<a href="https://www.ibm.com">IBM</a>`

PARTS OF AN HTML TAG

- **Tag name** → `a`
- **Opening tag** → `<a>`
- **Closing tag** → `</a>`
- **Attribute** → `href`
- **Attribute value** → `URL`
- **Content** → `IBM`

📌 ANALOGY:

- Tag name → Python class
- Individual tag → Object (instance)

HTML TREE (DOM STRUCTURE)

HTML documents form a **tree structure** (Document Object Model).

RELATIONSHIPS

- **Parent** → Tag containing another tag
- **Child** → Nested inside another tag
- **Siblings** → Same level tags
- **Descendants** → Any nested child at any level

EXAMPLE RELATIONSHIPS

- `<html>` is the **parent** of `<head>` and `<body>`
- `<head>` and `<body>` are **siblings**
- `<title>` is a **child** of `<head>`
- `<b>` is a **child** of `<h3>`

This tree structure is crucial for **BeautifulSoup navigation**.

HTML TABLES

TABLE STRUCTURE

```
<table>

  <tr>

    <th>Name</th>

    <th>Salary</th>

  </tr>

  <tr>

    <td>Player 1</td>

    <td>$10M</td>

  </tr></table>
```

TAGS USED

- `<table>` → Table container
 - `<tr>` → Row
 - `<th>` → Header cell
 - `<td>` → Data cell
-

5.29 WEB SCRAPING – INTRODUCTION

WHAT IS WEB SCRAPING?

Web scraping is the **automated process of extracting data from websites**.

WHY WEB SCRAPING?

- Manual copying is slow and inefficient
- Web scraping extracts **large data quickly**
- Essential for **data analysis & data science**

WEB SCRAPING WITH PYTHON

Main Libraries

- **Requests** → Downloads web pages
- **BeautifulSoup** → Parses and extracts data

BASIC WORKFLOW

1. Send HTTP request to website
2. Get HTML content
3. Parse HTML using BeautifulSoup
4. Extract required data

BASIC CODE EXAMPLE

```
from bs4 import BeautifulSoup
import requests

URL = "http://www.example.com"

page = requests.get(URL)

soup = BeautifulSoup(page.content, "html.parser")
```

BEAUTIFULSOUP OBJECTS

Tag Object

- Represents an HTML tag
- Example: title, h3, p

✚ If multiple tags exist, **first occurrence** is selected by default.

TREE NAVIGATION

Method	Purpose
.child	Move down the tree
.parent	Move up the tree
.next_sibling	Move sideways
.attrs	Access attributes
.string	Extract text

✓ PARSING HTML TAGS (TITLE, H3, CHILD, PARENT, SIBLING)

```
from bs4 import BeautifulSoup

html = """<!DOCTYPE html>

<html>

<head>

  <title>Page Title</title>

</head>

<body>

  <h3><b id='boldest'>Lebron James</b></h3>

  <p> Salary: $ 92,000,000 </p>

  <h3> Stephen Curry </h3>

  <p> Salary: $85,000,000 </p>

  <h3> Kevin Durant </h3>

  <p> Salary: $73,200,000 </p>

</body>

</html>"""

soup = BeautifulSoup(html, 'html5lib')
```

◆ ACCESS <TITLE> TAG

```
tag_object = soup.title
tag_object
```

✓ OUTPUT:

```
<title>Page Title</title>
```

◆ ACCESS FIRST <H3> TAG

```
tag_object = soup.h3
tag_object
```

✓ **OUTPUT:**

```
<h3><b id="boldest">Lebron James</b></h3>
```

◆ **ACCESS CHILD TAG ()**

```
tag_child = tag_object.b  
tag_child
```

✓ **OUTPUT:**

```
<b id="boldest">Lebron James</b>
```

◆ **ACCESS PARENT TAG**

```
parent_tag = tag_child.parent  
parent_tag
```

✓ **OUTPUT:**

```
<h3><b id="boldest">Lebron James</b></h3>
```

◆ **ACCESS NEXT SIBLING**

```
sibling_1 = tag_object.next_sibling  
sibling_1
```

✓ **OUTPUT:**

```
<p> Salary: $ 92,000,000 </p>
```

◆ **ACCESS ATTRIBUTES**

```
tag_child.attrs
```

✓ **OUTPUT:**

```
{'id': 'boldest'}
```

◆ **ACCESS TEXT INSIDE TAG**

```
tag_child.string
```

✓ **OUTPUT:**

```
'Lebron James'
```

5.30 find_all() METHOD

PURPOSE

- Filters HTML elements based on:
 - Tag name
 - Attributes
 - Text
 - Combination of filters

EXAMPLE

```
rows = table.find_all("tr")
```

- Returns a **list-like iterable**
- Each element is a **Tag object**

5.31 EXTRACTING TABLE DATA

```
for row in rows:  
    cells = row.find_all("td")  
    for cell in cells:  
        print(cell.text)
```

PARSING AN HTML TABLE

```
from bs4 import BeautifulSoup  
  
html = """<table>  
<tr><td>Pizza Place</td><td>Orders</td><td>Slices</td></tr>  
<tr><td>Domino's Pizza</td><td>10</td><td>100</td></tr>  
<tr><td>Little Caesars</td><td>12</td><td>144</td></tr></table>"""  
  
table = BeautifulSoup(html, 'html5lib')
```

◆ FIND ALL TABLE ROWS

```
table_rows = table.find_all(name='tr')  
  
table_rows
```

✓ OUTPUT:

```
[  
<tr><td>Pizza Place</td><td>Orders</td><td>Slices</td></tr>,  
<tr><td>Domino's Pizza</td><td>10</td><td>100</td></tr>,  
<tr><td>Little Caesars</td><td>12</td><td>144</td></tr>  
]
```

◆ LOOP THROUGH ROWS AND COLUMNS

```
for i, row in enumerate(table_rows):  
    print("Row", i)  
    cells = row.find_all("td")  
    for j, cell in enumerate(cells):  
        print("  Column", j, ":", cell.text)
```

✓ OUTPUT:

```
Row 0  
  Column 0 : Pizza Place  
  Column 1 : Orders  
  Column 2 : Slices  
Row 1  
  Column 0 : Domino's Pizza  
  Column 1 : 10  
  Column 2 : 100  
Row 2  
  Column 0 : Little Caesars
```

Column 1 : 12

Column 2 : 144

5.32 SCRAPING LIVE WEB PAGES

Steps

1. Import required libraries
2. Use requests.get(URL)
3. Use .text or .content
4. Create BeautifulSoup object
5. Extract data

WEB SCRAPING <a> TAGS FROM A WEBSITE

```
import requests

from bs4 import BeautifulSoup

page = requests.get("http://EnterWebsiteURL").text

soup = BeautifulSoup(page, "html.parser")

artists = soup.find_all('a')

for artist in artists:

    name = artist.contents[0]

    fullLink = artist.get('href')

    print(name)

    print(fullLink)
```

IMPORTANCE OF WEB SCRAPING IN DATA SCIENCE

Applications

-  Data Collection
-  Real-time updates (weather, prices)
-  Machine Learning training data

-  Market & trend analysis

OTHER WEB SCRAPING TOOLS

Scrapy

- Advanced crawling framework
- Used for large-scale scraping

Selenium

- Automates browsers
- Used for **dynamic websites** (JavaScript-heavy)

WEB SCRAPING TABLES USING PANDAS

read_html() Function

- Extracts **all tables** from a webpage

```
import pandas as pd

URL = "https://en.wikipedia.org/wiki/List_of_largest_banks"

tables = pd.read_html(URL)

df = tables[0]
```

ADVANTAGES

- Very fast
- Directly returns DataFrame

LIMITATIONS

- Extracts **only tabular data**
- Scrapes **hidden or unwanted tables**
- Retains hyperlinks → requires data cleaning

✦ **BeautifulSoup is still the most flexible method**

ETHICAL CONSIDERATIONS

- Respect website **terms of service**
- Follow **robots.txt**
- Avoid overloading servers

READING: WEB SCRAPING - A KEY TOOL IN DATA SCIENCE

READING: WEB SCRAPING TABLES USING PANDAS

HANDS-ON LAB: WORKING WITH DIFFERENT FILE FORMATS

5.33 WORKING WITH DIFFERENT FILE FORMATS – NOTES

1. INTRODUCTION

When working with data, it rarely comes in just one format. To build a **data-driven story or analysis**, you must know how to read and process **different file formats**.

After this topic, you should be able to:

- Identify common file formats (csv, json, xml)
- Read and output data using Python
- Know which Python libraries are used for each format
- Store extracted data using **DataFrames**

2. UNDERSTANDING FILE EXTENSIONS

A file extension appears at the **end of a filename** and tells us:

- What type of file it is
- Which software or library can open it

COMMON FILE FORMATS

Extension	File Type
.csv	Comma-Separated Values
.json	JavaScript Object Notation
.xml	Extensible Markup Language

Example:

FileExample.csv

➡ This tells us the file is a **CSV file**

3. PYTHON LIBRARIES FOR READING FILES

Key Libraries

File Type	Python Library
CSV	pandas
JSON	json
XML	xml (etree) / custom parsing

✦ **Pandas** is the most commonly used library for data analysis.

4. READING CSV FILES USING PANDAS

Why CSV?

- Simple
- Widely used
- Easy to import into Python

Steps to Read a CSV File

1. Import pandas
2. Assign file path to a variable
3. Use read_csv()

Example

```
import pandas as pd  
  
file = "FileExample.csv"  
  
df = pd.read_csv(file)  
  
df.head()
```

✦ read_csv() loads CSV into a DataFrame

✦ df.columns renames column headers

HANDLING MISSING HEADERS

✗ Problem:

- CSV file has **no column headers**
- Pandas treats **first row as header by default**

✓ SOLUTION:

- Manually assign column names using columns attribute

```
# Rename columns  
df.columns = ['Name', 'Phone Number', 'Birthday']  
df.head()
```

✦ This organizes the data neatly into labeled columns.

5. USING DATAFRAMES

- Pandas stores tabular data in **DataFrames**
- DataFrames allow:
 - Easy data manipulation
 - Clean output
 - Compatibility with analysis tools

6. READING JSON FILES

What is JSON?

- Language-independent data format
- Similar to a **Python dictionary**
- Used widely in APIs and web data

Steps to Read JSON

1. Import json
2. Open the file
3. Use json.load()
4. Print or process data

Example

```
import json

with open('filesample.json', 'r') as openfile:

    json_object = json.load(openfile)

json_object
```

- ✦ json.load() → JSON file → Python dictionary
- ✦ Use when JSON structure is **nested or non-tabular**
- ✦ JSON data is usually accessed using **keys**, just like dictionaries.

7. READING XML FILES

What is XML?

- **Extensible Markup Language**
- Data stored using tags (similar to HTML)
- Hierarchical structure

Important Point

- ✗ Pandas does **not** directly support XML reading
- ✓ XML must be **parsed manually**

Steps to Read XML

1. Import xml.etree.ElementTree
2. Parse the XML file
3. Traverse the document
4. Extract data
5. Store it in a DataFrame

Basic Example

```
import pandas as pd

import xml.etree.ElementTree as etree

# Parse XML file

tree = etree.parse("fileExample.xml")

root = tree.getroot()

# Define DataFrame columns

columns = ["Name", "Phone Number", "Birthday"]

rows = []
```

Extracting Data

- Loop through XML elements
 - Collect required values
 - Append them to a DataFrame
 - Assign column headers manually
- ✦ XML parsing requires **more effort** than CSV or JSON.

```
# Extract data

for node in root:

    name = node.find("name").text

    phonenumber = node.find("onenumber").text

    birthday = node.find("birthday").text

    rows.append([name, phonenumber, birthday])

# Create DataFrame

df_xml = pd.DataFrame(rows, columns=columns)

df_xml.head()
```

8. SUMMARY OF FILE FORMATS

File Type	Structure	Library Used	Difficulty
CSV	Tabular	pandas	Easy
JSON	Key-value	json	Medium
XML	Tree-based	xml.etree	Hard

HANDS-ON LAB: WORKING WITH DIFFERENT FILE FORMATS

PRACTICE PROJECT: GDP DATA EXTRACTION AND PROCESSING

CHEAT SHEET: APIS AND DATA COLLECTION

MODULE 5 GLOSSARY: APIS AND DATA COLLECTION
